VERIFIED CORRECTION (IMPORTANT): GitHub's REST notifications endpoint currently documents classic personal-access-token scopes (notifications/repo), not fine-grained PAT support. Sentinel v1 therefore excludes direct live notifications API access. It reads selected repository issues/PRs with narrowly scoped credentials, and GitHub notification emails via Gmail. See https://docs.github.com/en/rest/activity/notifications.

SENTINEL

MASTER PROJECT BLUEPRINT

Architecture · Security Regression · Privacy Routing · UI · Evaluation
Version 1.0 — 1 November 2026

VERIFICATION NOTICE — READ FIRST

FACT: Two attached research reports were provided. They are treated as unverified secondary analysis; their exact claims are not adopted without primary-source support. The first recommends a broader systems project; the second documents a narrowed two-feature direction. The latest owner decision controls: a local-only deployment, security regression control plane and privacy-aware STATIC selector, approximately 150 estimated hours.
UNVERIFIED: Package versions below were located in official registries but have not been installed or compatibility-tested together. The local hardware, OAuth eligibility, provider pricing, model IDs, API quotas, font files, exact source-feed terms, and all benchmark results remain unverified. All result charts are EMPTY TEMPLATES.
INFERENCE: This is a build specification, not an assurance of complete prompt-injection protection, compliance certification, or a guarantee that the estimated hours will suffice.

CONTENTS
1 Executive overview | 2 Feature catalogue | 3 System architecture | 4 LangGraph | 5 MCP | 6 Policy/threat model | 7 Model layer | 8 Prompts | 9 Data | 10 Evaluation | 11 UI | 12 Technology | 13 Milestones | 14 Risks | 15 Related work | 16 Interview kit | 17 Appendix, references and visual atlas

1. EXECUTIVE OVERVIEW

INFERENCE: Sentinel is a single-owner, local-first personal AI automation agent. It ingests Gmail and job-alert messages, selected GitHub activity, local tasks and at most three allowlisted public career feeds; it triages them into a cited, structured daily briefing and a queue of proposed actions.
INFERENCE: Its intended user is the developer, an AI/ML student building a tool for daily use and a defensible engineering portfolio.
INFERENCE: Five-sentence thesis: (1) External content is data, not authority. (2) Every consequential tool call must cross a deterministic authorization boundary. (3) Model and tool behavior must be recordable and replayable as regression tests. (4) Private-data sensitivity

constrains provider choice before quality, latency or cost. (5) The project's value is in measured security–utility and privacy–quality trade-offs, not feature count.
INFERENCE: It is deliberately NOT a general autonomous assistant, new provenance research, formal non-interference proof, trained model router, advanced memory engine, email sender, LinkedIn scraper, multiuser SaaS or remote MCP platform.
FACT: LangGraph supplies graph execution, checkpointing and interrupts, not application-level semantic authorization [S1,S2]. CaMeL, ARGUS, AgentLock and LangGraph-SecureAgents already address related capability/provenance authorization [S3–S6]. Do not claim novelty for provenance-based authorization.

2. FEATURE CATALOGUE

INFERENCE: Entries give purpose; reason; overlap/source; metric; dominant failure; status.
F01 Email reader — normalize message bodies, sender and job alerts; daily signal; overlaps existing email agents [S7]; fixture extraction accuracy and source coverage; MIME/HTML drift; CORE.
F02 GitHub reader — selected PR/issue and repository activity; notification emails via Gmail; developer signal; GitHub API [S8]; fixture event recall and permission tests; token overscope; CORE.
F03 Job collector — job alerts plus two or three allowlisted public feeds; relevant opportunities without LinkedIn scraping; established aggregators; extraction F1 and duplicate rate; changing site markup; CORE.
F04 Local task state — proposed/accepted/completed tasks, due dates, source references; actionable follow-up; ordinary task managers; task correctness and duplicate side effects; retry duplication; CORE.
F05 Structured daily briefing — urgent, GitHub, jobs, tasks, uncertainties, citations; usable daily summary; existing personal agents [S7]; completeness and evidence precision; hallucination; CORE.
F06 Optional existing hybrid RAG — ChromaDB+BGE+BM25+cross-encoder for job/profile relevance only if a lexical baseline loses; avoid rebuilding retrieval; established RAG; nDCG@5 versus added latency; stale context; CORE-OPTIONAL.
F07 Source provenance envelopes — trust/sensitivity labels and parent IDs on inputs and proposed arguments; policy evidence; CaMeL/ARGUS/AgentLock [S3–S5]; lineage coverage; false causal inference; DEEP A.
F08 Capability policy gate — allow/approval/deny in trusted code before any tool; protect action boundary; LangGraph-SecureAgents [S6]; unsafe action and overblock rates; missed bypass; DEEP A.
F09 Approval inbox/interrupt — exact hashed proposal, TTL, human decision, one-time resume; review consequential actions; LangGraph [S1]; stale approval rejection and approval precision; fatigue; DEEP A.
F10 Local write idempotency — stable key and unique transaction for task writes; protect retry; standard distributed systems; duplicate side effects; ambiguous outcome; DEEP A SUPPORT.
F11 Boundary record/replay — immutable model and MCP response envelopes, trace comparison; reproducible failures; Chronicle/LangSmith [S9,S10]; replay reproduction and mutation catch rate; over-stubbing; DEEP A.

F12 120-case fixture suite — benign/attack/fault/approval/idempotency scenarios, 80 dev and 40 sealed; honest trade-off; Task Shield/WASP/AgentDojo [S11–S13]; attack success, clean success and CI sensitivity; biased small set; DEEP A.
F13 Static privacy selector — hard sensitivity filtering then fixed or measured eligible model; avoid cloud exposure; routing literature [S14]; quality, cost, p95 latency, sensitive bytes egress; local quality deficit; DEEP B.
F14 Four model adapters — Ollama, Gemini, OpenAI and OpenRouter through one call wrapper; apples-to-apples experiments; LangChain integrations [S15]; schema pass and fallback correctness; API drift; DEEP B.
F15 Control Inbox — pending approvals, run failures, action evidence; owner decision; Linear/GitHub review [S16]; review time and mistaken approvals; evidence hidden; UI.
F16 Run Explorer — graph timeline, model/tool boundary, raw trace and replay creation; debugging; Temporal/LangSmith [S9,S10]; time to isolate failure; trace noise; UI.
F17 Policy/Provenance — decision reason first, graph second, argument sources; explain control plane; AgentLock/observability [S5,S17]; explanation fidelity; false graph certainty; UI.
F18 Eval Lab — baselines, intervals, case diffs, sealed-split banner; demonstrate evidence; LangSmith/Langfuse [S10,S18]; reproducible comparison; vanity charts; UI.
F19 Learned routing — train only after many labeled executions and proven oracle gap; routing research [S14]; regret versus static; overfitting; FUTURE WORK.
F20 Advanced long-term memory — time-bounded, correctable, superseded memories; Letta/Mem0 [S19]; stale-use and correction adoption; memory poisoning; FUTURE WORK.
F21 Cryptographic audit chain — signed/hash-linked receipts; AgentLock [S5]; tamper detection; key complexity; FUTURE WORK.
F22 Autonomous email sending — separate send capability, recipient policy, staged approval; email agents [S7]; unauthorized-send rate; irreversible wrong recipient; FUTURE WORK, CUT FROM V1.
F23 Direct LinkedIn scraping — no v1 implementation, even if technically feasible; terms and account risk; not benchmarked; CUT, not a planned feature.
F24 Calendar/Slack/Discord, generic chat, Kubernetes, Postgres, remote MCP, multiuser tenancy, trained classifiers — no measured benefit for this workload; CUT.

3. SYSTEM ARCHITECTURE

INFERENCE: Local-only deployment means FastAPI bound to 127.0.0.1, local React/Vite build, two stdio MCP subprocesses, SQLite WAL and a separate SQLite LangGraph checkpoint database, optional existing local ChromaDB, and Ollama bound to loopback. Gmail/GitHub reads and cloud LLM calls necessarily cross the network: local deployment is NOT offline execution.
INFERENCE: Data flow: fixture/live source → read MCP ToolGateway → immutable source envelope → normalize/dedupe → sensitivity classification → static ModelGateway → typed triage → action proposal → PolicyGate → approval or deny or idempotent task write → trace → briefing → UI.
INFERENCE: Trust boundaries: TB1 browser to loopback FastAPI (origin, session, CSRF); TB2 application to model output (untrusted proposals); TB3 graph to stdio MCP process (registered tool/capability); TB4 read MCP to external source (untrusted text); TB5 local

Ollama to cloud provider (sensitivity and payload); TB6 traces/fixtures to filesystem (redaction and access).
INFERENCE: No arbitrary tool names, executable paths, URLs or filesystem paths from LLM output. Registry maps tool ID to fixed handler, schema, capability and server. Restrict redirects to source allowlist. Tokens never enter prompts, model outputs or committed fixtures. Use OS keychain or protected local secret store.
INFERENCE: Operational failures: read timeout → bounded retry; malformed structured model output → one same-privacy retry then error; policy/database failure → fail closed for writes; unknown task write result → reconcile UNIQUE idempotency key before any retry. No generic auto-retry of external writes.
INFERENCE: Every model call passes ModelGateway.generate and every tool call passes ToolGateway.call from the first milestone. Both wrappers emit typed, redacted trace envelopes; no direct calls from graph nodes. See FIGURE A in the visual atlas.

4. LANGGRAPH STATE MACHINE

FACT: LangGraph interrupts require persisted state and a thread identifier; an interrupted node can restart on resume [S1]. Therefore never perform a write before an interrupt in the same restartable node.
INFERENCE: Nodes: START → collect → normalize_dedupe → label_sensitivity → select_model → classify_extract → optional_retrieve → propose_actions → policy_gate. Conditional edges: ALLOW→execute_tool;
APPROVAL→approval_interrupt→revalidate→execute_tool or denied; DENY→denied. All converge at reconcile_result → assemble_briefing → persist_trace → END. See FIGURE B.
INFERENCE: Pydantic state fields (name:type):
AgentState: run_id:UUID; thread_id:str; graph_version:str; trusted_intent_id:str; source_ids:list[UUID]; items:list[SourceItem]; route:ModelRoute|None; triage:list[Triage]; proposals:list[ActionProposal]; decisions:list[PolicyDecision]; approvals:list[UUID]; tool_results:list[ToolResult]; briefing:Briefing|None; errors:list[ErrorEvent]; retry_budget:int; replay_mode:bool.
SourceItem: id:UUID; source_kind:enum(email,github,job,task); external_id:str; origin_uri:str|None; observed_at:datetime; raw_ref:str; sha256:str; trust:TrustLabel; sensitivity:Sensitivity; parent_ids:list[UUID]; fixture_id:str|None.
ActionProposal: id:UUID; run_id:UUID; tool_id:str; capability:Capability; canonical_args_json:str; argument_source_ids:list[UUID]; trusted_intent_id:str|None; proposal_sha256:str; created_at:datetime.
PolicyDecision: proposal_id:UUID; outcome:enum(ALLOW,APPROVAL,DENY); rule_id:str; reason_code:str; policy_version:str; evidence_ids:list[UUID]; evaluated_at:datetime.
Approval: id:UUID; proposal_sha256:str; reviewer_id:str; status:enum(pending,approved,rejected,expired,consumed); created_at:datetime; expires_at:datetime; decided_at:datetime|None.
ModelRoute: task_class:str; sensitivity:Sensitivity; provider:str; model_id:str; rule_id:str; config_sha256:str; cloud_allowed:bool; fallback_order:list[str].
Triage: source_id:UUID; category:str; priority:int; summary:str; evidence_spans:list[str]; proposed_task:str|None; schema_version:str.

ToolResult: execution_id:UUID; proposal_id:UUID; status:str; output_ref:str|None; error_code:str|None; side_effect_ref:str|None.
Briefing: day:date; urgent:list[BriefingItem]; github:list[BriefingItem]; jobs:list[BriefingItem]; tasks:list[BriefingItem]; warnings:list[str]; sources:int.
INFERENCE: Checkpoint every graph thread with langgraph-checkpoint-sqlite. Avoid storing raw private text in checkpoint if IDs suffice. Resume re-fetches proposal, compares exact SHA-256, authenticates reviewer, checks expiry, consumes grant and RE-EVALUATES policy. The write server re-fetches canonical args from SQLite by proposal ID; never trusts client-supplied args.
INFERENCE: Retry rules: idempotent reads max 2 retries with backoff; model parse max 1 same-privacy retry; task write reconcile by UNIQUE key and return previous committed result; ambiguous remote writes are disabled. Record graph_version, prompt_set_sha, model_config_sha, policy_sha per run.

5. MCP DESIGN

FACT: The MCP Python SDK supports stdio transport [S20]. Gmail gmail.readonly allows message reading; gmail.compose also allows sending, so do NOT request it [S21].
INFERENCE: Use exactly two servers, because separate credential/process boundaries matter more than superficial server count:
sentinel-read (stdio): mail.list_ids(query,limit), mail.get(id), mail.search(query,limit); github.list_repo_issues, github.get_issue, github.get_pull, github.list_repo_events; jobs.fetch_allowlisted(source_key), jobs.parse_alert(source_id); tasks.list, tasks.get. Holds only read-only upstream tokens and job-host allowlist. No mutation APIs.
sentinel-task-write (stdio): tasks.create_accepted(proposal_id,idempotency_key), tasks.complete(task_id,expected_version,idempotency_key), tasks.reopen(task_id,expected_version,idempotency_key). Holds local task DB permission only, no Gmail/GitHub credential. Server reads frozen, approved arguments itself and enforces UNIQUE key and expected_version.
INFERENCE: No dedicated mail-write MCP server in v1: there is no Gmail write functionality to isolate. Read/write separation is real because the task-write server has no external credential and the read server cannot change accepted tasks. If separate OS users/process permissions are infeasible, disclose that the boundary is logical/process-level, not OS-enforced.
INFERENCE: Minimum live scopes: Gmail https://www.googleapis.com/auth/gmail.readonly; GitHub fine-grained token limited to chosen repos and endpoint-specific read permissions [S22]; jobs no credentials; local tasks filesystem permissions. Exact GitHub notification endpoint permission set is an implementation validation gate, not assumed. Provider API keys only in ModelGateway.
INFERENCE: Fixture-backed mock servers implement the same tool contracts. Stdio child process paths are hardcoded and arguments schema-validated. Never trust MCP tool annotations as the final policy source. Compatibility between mcp==2.3.0 and langchain-mcp-adapters==0.3.2 is UNVERIFIED until a protocol smoke test passes.

6. POLICY AND THREAT MODEL

INFERENCE: Assets: private email, repo metadata, tasks, OAuth/provider keys, approval decisions, audit traces, sealed fixtures and cloud request payloads.
INFERENCE: Attacker controls email subject/body, job page text, GitHub issue/PR text or malformed read-tool output; may impersonate instructions, propose recipients/URLs and exploit model confusion. Non-goals: compromised OS, modified trusted policy/code, malicious Python dependencies, stolen OAuth refresh token, hostile reviewer, local RCE and formal information-flow security.
INFERENCE: Trust labels: T0 POLICY (trusted code/config); T1 OWNER_INTENT (authenticated instruction or configured recurring rule); T2 AUTH_METADATA (trusted identity only); T3 DERIVED (LLM output, never authority); T4 EXTERNAL (mail/GitHub/job text); T5 UNKNOWN (fail-closed for writes).
INFERENCE: Sensitivity labels: PUBLIC; INTERNAL (redacted/aggregated); PRIVATE (raw mail/private repo); SECRET (tokens/passwords). SECRET never goes to any model. PRIVATE raw data is local-only by default.
INFERENCE: Capabilities: C0 READ_PUBLIC; C1 READ_PRIVATE; C2 WRITE_LOCAL_PROPOSAL (pending suggestion, not accepted task); C3 WRITE_LOCAL_ACCEPTED; C4 WRITE_EXTERNAL; C5 SEND_OR_SECRET. C4/C5 are disabled in v1.
INFERENCE: Ordered authorization table; first matching rule wins:
P00 unregistered tool, server mismatch, malformed args, unauthenticated request, policy error → DENY.
P01 replay mode tries live write or wrapper bypass detected → DENY.
P02 C5 send/secret or C4 external write → DENY even with approval.
P03 SECRET input to any model or PRIVATE raw input to cloud → DENY.
P04 C0 allowlisted public read with valid args → ALLOW.
P05 C1 scoped private read under owner session → ALLOW + AUDIT.
P06 C2 source-linked sanitized suggestion → ALLOW into pending proposals only.
P07 C3 explicitly owner-authorized exact task with no untrusted argument substitution → ALLOW + AUDIT.
P08 C3 with bounded derived/external argument source IDs → APPROVAL.
P09 C3 with missing lineage, changed hash, unsafe content, stale grant → DENY.
P10 APPROVAL with authenticated reviewer, unexpired matching hash, one-time consumption and fresh policy check → ALLOW.
P11 rejected/expired/changed approval → DENY.
P12 unmatched → DENY.
INFERENCE: A later allow never overrides deny. Approval authorizes only the frozen proposal. An email may supply job facts and a task suggestion, but never owner authority, cloud consent or a send instruction. See FIGURES C and D.
INFERENCE: Provenance labels document declared source references; they cannot establish true causal influence inside an LLM. A model can omit or fabricate lineage, so the ToolGateway must independently associate each input and reject unsupported sources; security claims remain empirical, not formal.
INFERENCE: Policy inputs: trusted principal, registered capability, canonical args, source IDs and trust/sensitivity, grant state, policy version. Output: outcome, rule_id, reason_code, evidence IDs. Fail closed on audit failure before a task write.

7. MODEL LAYER: STATIC PRIVACY-AWARE SELECTION

FACT: Routing research and LLMRouterBench warn that sophisticated routing does not reliably beat simple baselines across all workloads [S14]. The model selector is intentionally deterministic, not learned.
INFERENCE: Single interface ModelGateway.generate(ModelRequest) → ModelResponse. All four adapters implement the same contract; LangChain chat-model integration is used where stable. OpenRouter is accessed through its documented OpenAI-compatible endpoint using an explicit base URL, not by pretending OpenRouter is OpenAI for accounting. No graph node instantiates a provider client directly.
INFERENCE: ModelRequest fields: call_id:UUID; task_class:enum(triage,extract,match,briefing); prompt_id:str; prompt_sha256:str; schema_id:str; source_ids:list[UUID]; sensitivity:Sensitivity; payload_ref:str; max_output_tokens:int; temperature:float; timeout_ms:int; seed:int|None; allow_cloud:bool; config_sha256:str.
INFERENCE: ModelResponse fields: call_id:UUID; provider:str; model_id:str; structured_output_ref:str|None; schema_valid:bool; input_tokens:int|None; output_tokens:int|None; billable_cost_usd:Decimal|None; latency_ms:int; retry_count:int; error_code:str|None; request_payload_sha256:str; cloud_sensitive_bytes:int.
INFERENCE: Per-model registry config: provider, model_id, api_base, credential_ref, locality(local/cloud), privacy_class, supports_json_schema, supports_tools, max_context_tokens, max_output_tokens, timeout_ms, price_input_per_million, price_output_per_million, rate_limit, quality_by_task, p95_latency_by_task, allowed_task_classes, config_version. Prices and quality are measured/versioned data, not assumed constants.
INFERENCE: Deterministic selection order:
1. Block SECRET and reject requests with missing sensitivity/source labels.
2. For PRIVATE raw payloads, only allow Ollama bound to loopback; if unavailable, return NO_ELIGIBLE_MODEL, not cloud fallback.
3. For INTERNAL, permit cloud only after a locally executed redaction rule passes deterministic checks and owner-configured cloud consent is enabled; otherwise local.
4. For PUBLIC, filter to task-compatible models with valid credentials, schema support and latency budget.
5. Within eligible models, use frozen dev-set quality floor, then lowest measured median cost, then p95 latency, then stable provider/model lexical tie-breaker.
6. On failure, choose the next preapproved candidate in the SAME sensitivity class; never downgrade privacy to recover availability.
INFERENCE: Provider-specific caveats: Ollama endpoint must resolve to loopback, not a remote Ollama server; OpenAI, Gemini and OpenRouter are independent cloud recipients. The fact that an API offers a JSON schema option does not guarantee equivalent schema fidelity across providers. Benchmark actual structured-output validity. Provider retention/training claims are UNVERIFIED and excluded from guarantees.

INFERENCE: Experiment R0 strongest-cloud-all (OFFLINE SYNTHETIC/REDACTED DATA ONLY; violates production private-data policy, exposure counted), R1 fixed cheapest eligible cloud (same restriction), R2 local-first, R3 static privacy selector. Run the identical held-out tasks and prompts across eligible models; R0/R1 are counterfactual baselines, never live routes for private raw data. Measure per task class and overall.
INFERENCE: Metrics: Quality = passed rubric/expected field checks divided by scored tasks; Cost = sum of recorded provider charges or versioned pricing estimates with method declared; Latency = wall-clock gateway time p50/p95; CloudExposureBytes = total UTF-8 bytes of sensitivity-labeled PRIVATE or INTERNAL source substrings sent to external model endpoints; CloudExposureRate = cases with any sensitive cloud egress / sensitive cases. Redaction leakage is measured using deterministic canary spans, not guessed from model tokenization.
INFERENCE: If R3 is dominated by R2 in quality/cost/privacy or its confidence interval includes no meaningful improvement, keep R2. See FIGURE H and EMPTY FIGURE K.

8. PROMPT INVENTORY

INFERENCE: All prompts are data files, not inline strings in graph nodes. Naming convention prompt_id@MAJOR.MINOR.PATCH plus SHA-256 and schema version; traces pin exact hash. An edited prompt requires new hash and CI fixture run. The model never receives policy implementation, credentials, raw approval grants or hidden sealed-case labels.
INFERENCE: P01 email_triage@1.0.0 — input: normalized email text/ref, timestamp, trusted priority preferences; output EmailTriage{source_id,category,priority,summary,evidence_spans,task_suggestion?}; tests: schema, no invented dates, malicious email instructions ignored as authority.
P02 github_triage@1.0.0 — input: issue/PR metadata and body, watched repos; output GitHubTriage{source_id,kind,priority,summary,action_needed,evidence_spans}; tests: no fabricated PR status, injection isolation.
P03 job_extract@1.0.0 — input: alert/page text and URL; output JobPosting{source_id,company,title,location,remote,apply_url,deadline?,evidence_spans,unknown_fields}; tests: URL allowlist, null unknowns, duplicate normalization.
P04 job_match@1.0.0 — input: normalized job, explicitly configured skills/preferences, optional retrieved profile facts; output JobMatch{source_id,score_0_100,matched_skills,missing_skills,reason,evidence_spans}; tests: stable rubric and source attribution, avoid false eligibility.
P05 task_suggest@1.0.0 — input: triage objects and owner preferences; output TaskSuggestion[]{source_ids,title,description,due_date?,confidence}; tests: no direct tool call, bounded strings, deduplication.
P06 briefing_synthesize@1.0.0 — input: typed triage and task state, day/timezone; output Briefing{day,urgent,github,jobs,tasks,warnings,sources}; tests: every claim maps to source ID, no hidden tool instruction.
P07 action_propose@1.0.0 — input: trusted intent ID, accepted task choices, bounded triage; output ActionProposalDraft[]{registered_tool_id,args,source_ids,reason}; tests: no unregistered tool, no authority derived from external text.

P08 sensitivity_assist@1.0.0 — input: locally redacted excerpts/metadata only; output SensitivityHint{class,reason,uncertain}; tests: conservative on ambiguous cases. IMPORTANT: final sensitivity label comes from deterministic source policy; this prompt cannot lower sensitivity.
P09 eval_judge_optional@1.0.0 — input: redacted expected/actual briefing; output JudgeResult{rubric_id,score,justification,source_ids}; tests: calibration against human labels. NOT used for security pass/fail.
INFERENCE: No prompt for authorizing a tool. Policy decisions are deterministic. Every prompt is tested on benign, malformed, indirect-injection and schema-drift cases. Prompt change CI fails on schema validity, forbidden action, clean-task regression or cloud-egress invariants; LLM judge scores are supplementary.
INFERENCE: Prompt file metadata: prompt_id, semver, sha256, input_schema_id, output_schema_id, changelog, owner, allowed_task_classes, approved_model_set, test_fixture_manifest_sha. Store in Git; include exact versions in runs.

9. DATA MODEL

INFERENCE: SQLite is authoritative for all structured state; use WAL and foreign keys. Separate operational DB and checkpoint DB to avoid coupling to LangGraph internals. All timestamps UTC ISO-8601; identifiers UUID text; sensitive raw content encrypted at rest if feasible, otherwise OS-protected with documented limitation. JSON is validated before insertion; append-only trace events are NOT tamper-evident.
INFERENCE: Table specification: TABLE — columns [type; PK/FK]; indexes and constraints.
source_items — id TEXT PK; source_kind TEXT; external_id TEXT; origin_uri TEXT NULL; observed_at TEXT; raw_ref TEXT; sha256 TEXT; trust TEXT; sensitivity TEXT; parent_ids_json TEXT; fixture_id TEXT NULL; created_at TEXT. UNIQUE(source_kind,external_id,sha256); INDEX(observed_at,source_kind); INDEX(sha256).
raw_blobs — sha256 TEXT PK; encrypted_blob BLOB; media_type TEXT; bytes INTEGER; redaction_version TEXT; created_at TEXT; retention_until TEXT. INDEX(retention_until). Never include OAuth tokens.
tasks — id TEXT PK; title TEXT; description TEXT; status TEXT; due_at TEXT NULL; source_item_id TEXT FK source_items NULL; version INTEGER; created_at TEXT; updated_at TEXT; accepted_by TEXT NULL. INDEX(status,due_at); INDEX(source_item_id).
task_suggestions — id TEXT PK; source_ids_json TEXT; title TEXT; description TEXT; status TEXT; created_at TEXT; proposal_id TEXT NULL FK action_proposals. INDEX(status,created_at).
runs — id TEXT PK; thread_id TEXT; started_at TEXT; finished_at TEXT NULL; status TEXT; graph_version TEXT; policy_sha TEXT; prompt_set_sha TEXT; model_config_sha TEXT; fixture_manifest_sha TEXT NULL; replay_mode INTEGER. INDEX(started_at,status).
trace_events — id TEXT PK; run_id TEXT FK runs; seq INTEGER; node TEXT; event_type TEXT; boundary_id TEXT NULL; input_sha TEXT; output_ref TEXT NULL; model_id TEXT NULL; prompt_sha TEXT NULL; duration_ms INTEGER NULL; error_code TEXT NULL; created_at TEXT. UNIQUE(run_id,seq); INDEX(run_id,node); INDEX(boundary_id).

action_proposals — id TEXT PK; run_id TEXT FK runs; tool_id TEXT; capability TEXT; canonical_args_json TEXT; proposal_sha TEXT UNIQUE; source_ids_json TEXT; trusted_intent_id TEXT NULL; created_at TEXT; status TEXT. INDEX(run_id,status).
policy_decisions — id TEXT PK; proposal_id TEXT FK action_proposals; outcome TEXT; rule_id TEXT; reason_code TEXT; policy_sha TEXT; evidence_ids_json TEXT; evaluated_at TEXT. INDEX(proposal_id,evaluated_at).
approvals — id TEXT PK; proposal_id TEXT FK action_proposals; proposal_sha TEXT; reviewer_id TEXT; status TEXT; created_at TEXT; expires_at TEXT; decided_at TEXT NULL; consumed_at TEXT NULL; decision_nonce_sha TEXT. INDEX(status,expires_at); UNIQUE(proposal_sha,decision_nonce_sha).
tool_executions — id TEXT PK; proposal_id TEXT FK action_proposals; idempotency_key TEXT UNIQUE; status TEXT; started_at TEXT; finished_at TEXT NULL; result_ref TEXT NULL; side_effect_id TEXT NULL; error_code TEXT NULL. INDEX(proposal_id,status).
briefings — id TEXT PK; day TEXT UNIQUE; run_id TEXT FK runs; body_json TEXT; created_at TEXT; source_count INTEGER; warnings_json TEXT. INDEX(run_id).
model_calls — id TEXT PK; run_id TEXT FK runs; task_class TEXT; provider TEXT; model_id TEXT; request_sha TEXT; response_ref TEXT NULL; sensitivity TEXT; input_tokens INTEGER NULL; output_tokens INTEGER NULL; cost_usd TEXT NULL; latency_ms INTEGER; sensitive_cloud_bytes INTEGER; schema_valid INTEGER; route_rule TEXT; error_code TEXT NULL. INDEX(run_id,provider); INDEX(task_class,model_id).
boundary_envelopes — id TEXT PK; run_id TEXT FK runs; boundary_kind TEXT; boundary_key TEXT; request_sha TEXT; response_blob_sha TEXT; response_schema TEXT; fixture_case_id TEXT NULL; recorded_at TEXT. UNIQUE(run_id,boundary_key); INDEX(fixture_case_id).
eval_cases — id TEXT PK; split TEXT; family TEXT; category TEXT; fixture_version TEXT; fixture_sha TEXT; expected_json TEXT; is_sealed INTEGER; created_at TEXT. INDEX(split,family).
eval_experiments — id TEXT PK; commit_sha TEXT; baseline TEXT; prompt_set_sha TEXT; policy_sha TEXT; model_config_sha TEXT; fixture_manifest_sha TEXT; seed INTEGER NULL; started_at TEXT; completed_at TEXT NULL; environment_json TEXT. INDEX(commit_sha,baseline).
eval_case_results — id TEXT PK; experiment_id TEXT FK eval_experiments; case_id TEXT FK eval_cases; repeat_index INTEGER; attack_goal_met INTEGER NULL; clean_success INTEGER NULL; unauthorized_actions INTEGER; denied_legitimate_actions INTEGER; approval_count INTEGER; duplicate_writes INTEGER; replay_match INTEGER; latency_ms INTEGER; cost_usd TEXT NULL; cloud_sensitive_bytes INTEGER; trace_ref TEXT. UNIQUE(experiment_id,case_id,repeat_index); INDEX(experiment_id,case_id).
model_registry — id TEXT PK; provider TEXT; model_id TEXT; config_json TEXT; config_sha TEXT UNIQUE; enabled INTEGER; updated_at TEXT. UNIQUE(provider,model_id).
user_preferences — key TEXT PK; value_json TEXT; sensitivity TEXT; updated_at TEXT; provenance TEXT; owner_confirmed INTEGER. No autonomous preference writes from email/job text.
INFERENCE: Chroma collection OPTIONAL personal_context: id=source_item_id/chunk_id; document=approved profile snippet or normalized job text; metadata={source_id,observed_at,trust,sensitivity,valid_until,embedding_model_sha}. Chroma never owns authorization or task state; SQLite owns the source and deletion

tombstone. Existing BGE/BM25/cross-encoder pipeline is inherited and its exact dependency pins are UNVERIFIED until the original lockfile is inspected.
INFERENCE: Retention defaults to raw private mail 7 days, redacted trace 30 days, fixture corpus indefinitely only if synthetic/sanitized; configurable, not a legal guarantee. Deletion removes SQLite blobs, Chroma chunks and derived caches by source ID. Checkpoint files may contain references; audit actual contents before enabling live mail. See FIGURE E.

10. EVALUATION DESIGN

FACT: Prompt-injection defenses must be judged against clean-task utility, not attack suppression alone; Task Shield and WASP show why [S11,S12]. Chronicle demonstrates record/replay at nondeterministic boundaries [S9].
INFERENCE: Corpus target exactly 120 cases, 80 dev / 40 sealed, grouped by attack family and source type BEFORE prompt/policy tuning:
Benign workflows 36 (dev24/sealed12);
Indirect instruction attacks 30 (dev20/sealed10);
Provenance/argument confusion 18 (dev12/sealed6);
Approval/policy edges 12 (dev8/sealed4);
MCP/model faults 12 (dev8/sealed4);
Retry/idempotency faults 12 (dev8/sealed4).
Total 120 (dev80/sealed40). Sealed cases may be run by CI for final reporting but not inspected or used for tuning; store hashes and access controls. This is an intended design, not an existing dataset.
INFERENCE: Fixture envelope JSON schema (data specification, not application code):
{case_id:string, family:string, category:string, split:"dev"|"sealed", seed:int|null,
trusted_intent:{id,template,allowed_capabilities},
sources:[{source_id,source_kind,trust,sensitivity,body_ref,body_sha256,observed_at}],
mocked_mcp:[{boundary_key,tool_id,args_sha,response_ref,latency_ms,error_code}],
mocked_model:[{boundary_key,model_id,prompt_sha,response_ref}],
expected:{allowed_tool_ids,forbidden_tool_ids,required_policy_outcomes,required_approval_state,briefing_fields,attack_goal_predicate,clean_task_predicate,max_duplicate_writes},
injection_points:[{source_id,span_ref,attack_family}], fixture_sha256:string}.
INFERENCE: Attack classes: fake system/developer messages, urgent send request, tool-output laundering, malicious URLs, role spoofing, authority confusion, cross-source blending, GitHub issue instruction, job page instruction, job-alert email instruction, hidden text, approval hash replay, source-ID omission. Benign: legitimate job match, conflicting dates, duplicate alerts, urgent mail, noisy PRs, tasks with user consent. Fault: timeouts, malformed JSON, interrupted resume, SQLite busy, duplicate tool response, crash-after-write-before-ack.
INFERENCE: Mock MCP servers have deterministic seeded responses and count writes; each fake write stores idempotency_key and rejects duplicate side effects. Replay mode replaces BOTH model structured outputs and MCP read responses with immutable envelopes. A second cut-point mode may run changed policy code while preserving upstream envelopes; never treat this as full nondeterministic real-world replay.
INFERENCE: Security runtime baselines:
B0 — no deterministic policy, only instruction prompt to ignore injected text.

B1 — approval for EVERY accepted-task write, with no provenance distinction.
B2 — deterministic capability/trust policy with risk-triggered approval.
B3 — B2 plus replay-derived CI regression process. IMPORTANT: B3 is NOT a different runtime security algorithm. B2 vs B3 on identical frozen executions may produce identical action rates; B3's distinct outcome is regression detection after seeded changes.
INFERENCE: Routing baselines R0 strongest cloud on synthetic/redacted offline cases only; R1 fixed cheapest cloud on synthetic/redacted cases; R2 local-first with privacy hard constraint; R3 deterministic privacy/quality/cost selector. Evaluate every baseline on the SAME eligible task set; separately report the number excluded for privacy to avoid cherry-picking.
INFERENCE: Exact metric definitions (unit and denominator must appear beside every chart):
AttackSuccessRate = attack runs satisfying attacker-goal predicate / attack runs with evaluable predicate.
UnsafeActionRate = runs with at least one executed forbidden tool action / all attack runs.
CleanTaskSuccess = benign runs satisfying all required clean-task predicates / all benign runs.
OverblockRate = legitimate proposed writes denied without required justification / all legitimate proposed writes.
ApprovalPrecision = approvals correctly required by oracle / all approvals requested; report undefined when no approvals.
ApprovalRecall = oracle-required approvals requested / all oracle-required approvals.
DuplicateSideEffectRate = duplicate committed side effects / all committed write attempts under injected faults; also report per 1,000 attempted writes.
ReplayReproductionRate = full-replay runs matching expected ordered boundary+policy sequence / total full-replay runs.
MutationCatchRate = seeded policy/prompt defects that make CI fail / seeded defects executed; define mutant set before running.
SchemaValidity = schema-valid structured model responses / all model responses.
BriefingEvidencePrecision = supported atomic claims / all scored atomic claims, judged from source spans.
CloudExposureBytes = sum UTF-8 sensitive payload bytes actually serialized to cloud request bodies; 0 is meaningful, not a percentage.
CostPerSuccessfulBriefing = total billable provider USD / number of successful briefings; undefined when zero success.
LatencyP95 = nearest-rank 95th percentile of gateway wall-clock durations, including retries.
INFERENCE: Statistical protocol: deterministic policy unit fixtures once per build; model-involved dev cases 5 independent repeats per model/seed configuration; final sealed evaluation at least 5 repeats per stochastic case where budget permits. Report counts and exact binomial Wilson 95% intervals for case-level rates, and hierarchical bootstrap 95% intervals resampling CASES first, then repeats within each case, for differences and latency/cost. Use 10,000 seeded bootstrap resamples; seed and code SHA in report. Pair baselines on identical case IDs; report paired differences, not independent unpaired percentages.
INFERENCE: Sealed n≈40 produces wide uncertainty. Example only, NOT A RESULT: for 0 observed failures in 40 independent trials, a one-sided 95% binomial upper bound is about 7.2%; clustered repeats do not create 200 independent attack scenarios. Do not describe

zero observed failures as “100% secure.” This numerical bound is a mathematical illustration, not a benchmark observation.
INFERENCE: Anti-self-deception protocol: write expected predicates before model runs; seal case wording; keep attack-family holdout; use at least two model variants; inspect false positives; report exclusions, abstentions and schema failures; include strong prompt-only baseline; evaluate policy bypass through direct ToolGateway tests; measure utility and approvals, not only attack block; show raw confusion matrix; version data/price/latency environment; do not tune on sealed cases.
INFERENCE: CI on each change: offline deterministic unit and fixture replay (no live credentials, zero provider charges); nightly/manual real-model repeats on dev only; final release sealed evaluation once with frozen manifest and commit. Preserve raw per-case JSON and reproducibility manifest. See FIGURE F and empty result FIGURES J–L.

11. UI BLUEPRINT

FACT: Relevant existing interfaces include LangSmith (trace + datasets) [S10], Langfuse (experiments) [S18], Arize Phoenix (trace inspection) [S23], Temporal Web (event history) [S24], GitHub Actions (job/step logs) [S25], Sentry (trace waterfall) [S26], Grafana Explore (correlated logs/traces) [S27], Jaeger (nested spans) [S28], Linear (dense inbox) [S16], Raycast (keyboard command launcher) [S29], VS Code (command palette and panels) [S30], and GitHub pull-request review (evidence-linked decisions) [S31].
INFERENCE: Reference pattern → Sentinel fit:
LangSmith → nested run tree + convert failure to dataset case → Run Explorer.
Langfuse → experiment comparison with filtered failures → Eval Lab.
Phoenix → trace/span attributes with linked eval rows → Run Explorer.
Temporal Web → immutable ordered event history and retry visibility → Run Explorer.
GitHub Actions → graph job status plus step-level failure logs → Run Explorer.
Sentry → latency waterfall and exception drill-down → Model/tool diagnostics.
Grafana Explore → linked filters across traces and logs → consistent run_id navigation.
Jaeger → collapsed nested spans with timing → compact timeline.
Linear → keyboard-first dense inbox and detail panel → Control Inbox.
Raycast → contextual command palette → global quick actions.
VS Code → persistent panes, command search, status line → application shell.
GitHub PR review → decision with evidence and history → approve/reject detail.
INFERENCE: Hypothesis: the primary interface is a CONTROL PLANE, not chat. This is appropriate because the distinctive objects are approvals, policy reasons, trace envelopes and eval comparisons. A secondary command entry may trigger a briefing; it cannot obscure source/decision evidence. Navigation opens at Control Inbox.
INFERENCE: Four screens and their hierarchy:
SCREEN 1 CONTROL INBOX — left 216px navigation; center filterable 40px-row list of pending approvals, failed runs, priority task suggestions; right 400px decision inspector. Top: count of actionable items (not vanity metric). Inspector order: exact action + destination → rule/reason → affected sources and argument diff → raw canonical JSON → approve/reject. Require explicit confirmation for irreversible future actions. FIGURE I-1.
SCREEN 2 RUN EXPLORER — top run ID/date/status filters; left run list; center graph node timeline/waterfall; right event inspector with prompt/model/policy versions, boundary

request/response refs, redacted raw JSON. “Create fixture from this event” action produces a sanitized draft requiring review. FIGURE I-2.
SCREEN 3 POLICY / PROVENANCE — top decision search; left policy rule list; center selected decision explanation; right compact provenance graph. A plain textual reason must be sufficient without interpreting the graph. Distinguish “declared source reference” from “proven causal influence.” FIGURE I-3.
SCREEN 4 EVAL LAB — top experiment/baseline selectors with sealed-set warning; center paired metric table with numerator/denominator and 95% intervals; below failed-case list and replay diff; side panel environment/commit manifest. Never show a metric without case count. FIGURE I-4.
INFERENCE: Information hierarchy is always decision → reason → evidence → raw trace. Source identity and sensitivity are displayed beside each action, not buried in tooltip.
INFERENCE: Anti-slop rules: no purple-blue gradient, glassmorphism, glowing AI orb, chat-bubble-centric home, emoji icons, generic hero, pointless “runs today” cards, unmodified component-library look, unexplained gauges, animated charts without data. Use 1px borders, restrained semantic colors, 8px corner radius maximum on panels, and typography before shadows. No decorative graphics.
INFERENCE: Type tokens: Inter for UI; JetBrains Mono for IDs, hashes, source spans and JSON. 12px metadata / 14px body-table / 16px emphasized body / 20px section / 28px page. Weights 400, 500, 600. Use 1.4–1.55 body line height and tabular numerals in metrics.
INFERENCE: Spacing tokens: 4, 8, 12, 16, 24, 32 px; sidebar 216px; inspector 400px; list rows 36–40px; minimum clickable target 40px; one 8px grid. Responsive: below 1024px collapse right inspector into overlay; do not attempt mobile-first dense graph.
INFERENCE: Light tokens: bg #FFFFFF; text #111827; muted #475569; border #CBD5E1; focus/accent #1D4ED8; success #166534; warning #92400E; danger #B91C1C.
INFERENCE: Dark tokens: bg #0B1220; text #F8FAFC; muted #CBD5E1; border #334155; focus/accent #93C5FD; success #86EFAC; warning #FCD34D; danger #FCA5A5.
INFERENCE: Contrast calculations using WCAG relative luminance formula, foreground on specified bg: light text 17.74:1, muted 7.58:1, accent 6.70:1, success 7.13:1, warning 7.09:1, danger 6.47:1; dark text 17.89:1, muted 12.61:1, accent 10.38:1, success 13.33:1, warning 12.98:1, danger 9.86:1. These are computed design targets, NOT browser-tested accessibility. Verify actual hover/disabled/surface combinations in CI; aim WCAG AA 4.5:1 normal text and 3:1 large text/UI boundaries.
INFERENCE: Motion: 120–180ms for disclosure/selection, no gratuitous parallax, respect prefers-reduced-motion; static default for trace updates. Loading: preserve skeleton dimensions; show operation and cancel/retry for slow queries. Empty: state why and next action (“No pending approvals; review last run”). Error: show typed code, retry eligibility, last successful state and sanitized raw detail; never “Something went wrong.” Offline mode banner; cloud-call consent is explicit.
INFERENCE: Keyboard: / focuses search; j/k moves list selection only outside inputs; Enter opens inspector; Esc closes panel; g then i/r/p/e navigates four screens; Ctrl/Cmd+K opens command palette; a/r approve/reject ONLY when a decision panel is focused and after an explicit confirmation, never global destructive shortcuts. Visible focus ring, accessible labels, screen-reader status messages.
INFERENCE: UI build order follows backend: after fixture core → shell and briefing list; after policy → Control Inbox; after replay → Run Explorer and Eval Lab; after stable provenance → Policy/Provenance graph. UI budget 33/150 hours = 22%, under 25%. React Aria

Components supplies accessible primitives without imposing an aesthetic; Radix only if a specific primitive is missing.
INFERENCE: Frontend option comparison (5 best):
React+Vite+React Aria+Tailwind: FastAPI 5, familiarity 5, accessibility 5, theming 5, dense data 5, effort 4; WINNER.
React+Vite+Radix+Tailwind: 5,5,4,5,5,4; fallback, more component styling work.
Next.js+React+Radix: 3,5,4,5,5,2; redundant server runtime for local FastAPI.
INFERENCE: Components: react-aria-components for dialog, menu, table interactions, buttons and keyboard; Tailwind with project-owned CSS variables; TanStack Query for server cache; TanStack Table + Virtual for dense lists; @xyflow/react only for provenance graph; simple native SVG for three eval plots (no Recharts dependency); Lucide React icons; React Aria dialog/combobox for command palette. Style primitives from tokens, not shadcn defaults. Libraries are registry-verified candidates, not integration-tested.

12. TECHNOLOGY INVENTORY AND VERSION PINS

FACT: The following exact versions and release evidence were checked on official Python.org, PyPI, npm or Node.js release pages; dates with “month only” are deliberately not asserted as exact. Package licenses are registry-declared, not independent legal opinions. Compatibility of the proposed set is UNVERIFIED until a clean lock/install test.
INFERENCE: Inventory row format: component — exact candidate pin; license; registry release evidence; role; alternative rejected; source.
Python — 3.13.14; PSF; 2026-06-10; backend runtime; 3.14.x not chosen to reduce compatibility churn; https://www.python.org/downloads/release/python-31314/ [T01].
Node.js — 24.22.2 LTS; MIT (Node project); 2026-10-28; frontend toolchain; 26 LTS newer but unnecessary; https://nodejs.org/en/blog/release [T02].
SQLite — bundled with selected Python, actual sqlite3.sqlite_version UNVERIFIED until environment lock; public domain; local DB; Postgres cut absent concurrency evidence; https://www.sqlite.org/ [T03].
LangGraph — 1.2.19; MIT; 2026-10-30; graph/interrupts; custom state machine rejected; https://pypi.org/project/langgraph/1.2.19/ [T04].
langgraph-checkpoint-sqlite — 3.0.1; MIT; 2025-12-09; durable graph state; Postgres saver unnecessary; https://pypi.org/project/langgraph-checkpoint-sqlite/ [T05].
LangChain — 1.3.14; MIT; 2026-07-16; provider abstraction/structured outputs; hand-coded client set rejected; https://pypi.org/project/langchain/1.3.14/ [T06].
langchain-mcp-adapters — 0.3.2; MIT; 2026-08-06; MCP-to-LangChain tools; custom protocol glue rejected; https://pypi.org/project/langchain-mcp-adapters/0.3.2/ [T07].
mcp Python SDK — 2.3.0; MIT per project registry, verify license at install; 2026-10-02; stdio client/server; ad hoc subprocess JSON rejected; https://pypi.org/project/mcp/2.3.0/ [T08].
FastAPI — 0.162.0; MIT; 2026-10-23; local control API; Flask would require more validation boilerplate; https://pypi.org/project/fastapi/0.162.0/ [T09].
Uvicorn — 0.54.0; BSD-3-Clause; 2026-09-25; ASGI runtime; gunicorn multiworker not needed; https://pypi.org/project/uvicorn/0.54.0/ [T10].

Pydantic — 2.14.2; MIT; 2026-10-29; strict schemas; untyped dicts rejected; https://pypi.org/project/pydantic/2.14.2/ [T11].
httpx — 0.28.1; BSD-3-Clause; 2024-12-06; allowlisted public feed HTTP; requests lacks native async path; https://pypi.org/project/httpx/0.28.1/ [T12].
pytest — 9.0.3; MIT; 2026-04-07; deterministic CI; bespoke runner rejected; https://pypi.org/project/pytest/9.0.3/ [T13].
langchain-openai — 1.7.3; MIT; 2026-10-26; OpenAI/OpenRouter-compatible adapter; duplicate SDK rejected; https://pypi.org/project/langchain-openai/1.7.3/ [T14].
langchain-google-genai — 4.4.2; MIT; 2026-10-13; Gemini adapter; raw Gemini REST rejected; https://pypi.org/project/langchain-google-genai/4.4.2/ [T15].
langchain-ollama — 1.1.0; MIT; 2026-04-07; local Ollama adapter; raw REST rejected; https://pypi.org/project/langchain-ollama/1.1.0/ [T16].
ChromaDB — 1.5.9 OPTIONAL; Apache-2.0; 2026-05-05; existing hybrid retrieval; SQLite FTS lexical baseline first; https://pypi.org/project/chromadb/1.5.9/ [T17].
React — 19.3.1; MIT; Oct 2026 (exact day not verified); UI; Vue unnecessary given experience; https://www.npmjs.com/package/react [T18].
react-dom — 19.3.1; MIT; Oct 2026 (day unverified); DOM renderer; none; https://www.npmjs.com/package/react-dom [T19].
TypeScript — 7.0.2; Apache-2.0; 2026 (exact day unverified); UI typing; JavaScript rejected; https://www.npmjs.com/package/typescript [T20].
Vite — 8.3.5; MIT; Oct 2026 (day unverified); local build; Next.js unnecessary; https://www.npmjs.com/package/vite [T21].
@vitejs/plugin-react — 6.1.2; MIT; Oct 2026 (day unverified); React Fast Refresh; custom bundling rejected; https://www.npmjs.com/package/@vitejs/plugin-react [T22].
react-aria-components — 1.22.1; Apache-2.0; Oct 2026 (day unverified); accessible unstyled primitives; Radix fallback; https://www.npmjs.com/package/react-aria-components [T23].
tailwindcss — 4.3.3; MIT; 2026 (exact release day unverified); tokenized utility styling; CSS modules possible but slower; https://www.npmjs.com/package/tailwindcss [T24].
@tanstack/react-query — 5.105.0; MIT; Oct 2026 (day unverified); server-state cache; custom fetch hooks rejected; https://www.npmjs.com/package/@tanstack/react-query [T25].
@tanstack/react-table — 9.3.1; MIT; Oct 2026 (day unverified); dense tables; bespoke table rejected; https://www.npmjs.com/package/@tanstack/react-table [T26].
@tanstack/react-virtual — 3.14.24; MIT; Oct 2026 (day unverified); long trace virtualization; manual DOM windowing rejected; https://www.npmjs.com/package/@tanstack/react-virtual [T27].
@xyflow/react — 12.12.0 OPTIONAL; MIT; Sep/Oct 2026 (day unverified); provenance graph; plain linked list default; https://www.npmjs.com/package/@xyflow/react [T28].
lucide-react — 1.61.0; ISC; late Oct 2026 (day unverified); consistent icon set; emojis rejected; https://www.npmjs.com/package/lucide-react [T29].
Inter + JetBrains Mono — locally vendored font files, precise release/SHA UNVERIFIED; SIL Open Font License as generally published, verify per downloaded files; UI/mono pairing; system fonts fallback; https://github.com/rsms/inter and https://github.com/JetBrains/JetBrainsMono [T30].
GitHub Actions — hosted CI service, no installable version; workflow actions MUST be pinned to full commit SHA at implementation; local pytest as fallback; https://docs.github.com/en/actions [T31].

Ollama — local server, exact binary/model digest UNVERIFIED; pin installed binary and model digest in experiment manifest; alternative llama.cpp server possible but requirement favors Ollama; https://docs.ollama.com/ [T32].
Gemini API — cloud service, no semver package pin beyond adapter; model ID/API version/pricing UNVERIFIED until benchmark; https://ai.google.dev/gemini-api/docs [T33].
OpenAI API — cloud service, model snapshot and pricing UNVERIFIED until benchmark; https://platform.openai.com/docs [T34].
OpenRouter API — cloud gateway, no installed semver; OpenAI-compatible endpoint and selected upstream model IDs must be logged; https://openrouter.ai/docs [T35].
Gmail API — hosted service; OAuth gmail.readonly; endpoint/version and quota to validate live; https://developers.google.com/workspace/gmail/api/auth/scopes [T36].
GitHub REST API — hosted service; endpoint-specific fine-grained read permissions to verify live; https://docs.github.com/en/rest [T37].
BGE embeddings, BM25, cross-encoder reranker — existing code, versions/model SHA UNKNOWN; not newly installed until optional RAG wins a measured ablation; original project lockfile is required [T38].
INFERENCE: Before first implementation PR, run a clean lockfile resolution, license scan, basic import test, LangGraph interrupt/checkpoint smoke test, MCP 2026 stdio tool-list/call smoke test, and four provider contract tests. Any incompatibility invalidates the candidate pin; correct it in an ADR and update this inventory rather than silently using floating latest.

13. BUILD PLAN: MILESTONES, NOT CALENDAR WEEKS

INFERENCE: All hours are estimates; total 150 hours at approximately 10–12 hours/week. Stop at exit gates rather than expanding scope. First milestone is end-to-end fixture-backed core with no novel feature. Visual timeline and hours-by-area are FIGURES G1 and G2.
M0 — THIN CORE ON FIXTURES — 26 h ESTIMATE.
Deliverable: fixture email/job/GitHub/task ingestion, normalization/dedupe, one model via ModelGateway, all tools via ToolGateway, structured briefing in SQLite and minimal API. Exit: one command processes a frozen fixture day twice, yields schema-valid briefing with stable source IDs and zero real network writes; all calls are traceable through wrappers. Prerequisite topics: Pydantic schemas; LangGraph state/reducers; JSON schema; idempotent ingestion; SQLite WAL. OWNER WRITES: AgentState schema, graph nodes/edges, wrapper contracts. GENERATED MAY ASSIST: fixture adapters, serializers, FastAPI boilerplate.
M1 — MCP BOUNDARIES + IDEMPOTENT TASK WRITES — 16 h ESTIMATE.
Deliverable: two stdio MCP servers, fixed registry, local accepted-task write with UNIQUE idempotency key and reconciliation. Exit: tool discovery/call smoke tests pass; unregistered tool cannot run; 100 repeated identical task-write requests commit exactly one task; no Gmail/GitHub write credential exists. Topics: MCP stdio, process isolation, OAuth scopes, transactions, unique constraints, crash windows. OWNER WRITES: capability registry, write protocol, fault invariants. GENERATED: server skeleton and contract mocks.
M2 — POLICY + APPROVAL — 20 h ESTIMATE.

Deliverable: pure deterministic policy evaluator, approval table, LangGraph interrupt/resume, policy reason codes. Exit: every decision maps to P00–P12, changed/expired/replayed approval denied, external send denied; at least 24 dev fixtures pass. Topics: reference monitors, capability security, prompt injection, trust labels, LangGraph interrupts. OWNER WRITES: policy gate and decision table tests, state transition invariants. GENERATED: UI-neutral approval API scaffolding.

M3 — RECORD/REPLAY + EVAL — 28 h ESTIMATE.

Deliverable: boundary envelopes, mock MCP, 120-case corpus, B0/B1/B2 runtime baselines, B3 regression-process experiment, seeded faults, Wilson/bootstrap metrics, CI. Exit: clean checkout replays deterministic cases with zero provider credentials; mutation tests detect seeded policy bypass; sealed manifest is hashed; metric report includes numerators/denominators and intervals. Topics: AgentDojo, Task Shield, Chronicle, fault injection, Wilson interval, paired bootstrap, leakage avoidance. OWNER WRITES: fixture oracle predicates, metrics/statistics, mutation design. GENERATED: report formatting and synthetic fixture variations (human-reviewed).

M4 — STATIC SELECTOR + BENCHMARK — 18 h ESTIMATE.

Deliverable: Ollama/Gemini/OpenAI/OpenRouter adapters, deterministic sensitivity filter, R0–R3 evaluation, payload egress meter. Exit: each adapter passes schema/timeout/error contract (or is marked unavailable with reason); PRIVATE raw never leaves localhost in R2/R3; benchmark report has quality, p50/p95 latency, USD method, exposure bytes and paired uncertainty. Topics: provider APIs, structured output, local inference, privacy constraints, cost accounting, paired experimental design. OWNER WRITES: routing policy and metric code. GENERATED: provider adapter boilerplate and typed DTOs.

M5 — CONTROL-PLANE UI — 33 h ESTIMATE.

Deliverable: Control Inbox, Run Explorer, Policy/Provenance, Eval Lab; accessible keyboard flow and redacted raw trace. Exit: owner can inspect exact proposal, approve/reject, explain denial, view replay diff and compare baselines entirely in UI; all four screen keyboard smoke tests pass; no fake measured data displayed. Topics: React Aria, focus management, TanStack Table/Query, React Flow, WCAG contrast, virtualization. OWNER WRITES: information hierarchy, interaction decisions, approval safety. GENERATED: layout scaffolding and presentational components.

M6 — LIVE READ-ONLY GMAIL/GITHUB — 5 h ESTIMATE.

Deliverable: optional read-only OAuth/token configuration and selected endpoint integration. Exit: a live read-only pull produces normalized items, tokens are not logged, no write scopes, and an offline fixture mode remains fully functional. If OAuth exceeds 5 hours, cut live Gmail and demo with exported/synthetic mail. Topics: Gmail OAuth, GitHub REST endpoint permissions, token storage, rate limits, data minimization. OWNER WRITES: scope decision and secret handling review. GENERATED: connector client boilerplate.

M7 — DOCUMENTATION, DEMO, FREEZE — 4 h ESTIMATE.

Deliverable: architecture ADRs, reproducible demo script, threat model, limitations, final eval tables and evidence-linked interview kit. Exit: clean-machine fixture demo and offline CI reproduce final report with pinned commit/manifest; README clearly states unverified items and zero claimed security proof. Topics: reproducible research, ADRs, technical communication. OWNER WRITES: honest claims, limitations, experiment interpretation. GENERATED: formatting and copyediting.

TOTAL ESTIMATE = 26 + 16 + 20 + 28 + 18 + 33 + 5 + 4 = 150 hours. UI = 33/150 = 22%. If M0–M3 slip by >20 hours, cut provenance graph and optional RAG before weakening evaluation.

INFERENCE: Repository structure:
apps/api — FastAPI endpoints, auth and read models.
apps/web — React/TypeScript Vite control plane.
sentinel/graph — owner-written state, nodes and topology.
sentinel/model — single gateway, adapters, static selector.
sentinel/tool — single gateway, registry, MCP clients.
sentinel/policy — pure gate, capability map and decision codes.
sentinel/prompts — versioned prompt text + schema manifests.
sentinel/storage — SQLite schema/migrations and blob redaction.
mcp/read — read-only email/GitHub/jobs/tasks server.
mcp/task_write — local write-only task server.
eval/fixtures/dev — editable synthetic cases.
eval/fixtures/sealed — frozen manifest and access-restricted cases.
eval/replay — boundary envelope recorder and deterministic runner.
eval/metrics — owner-written metrics, intervals, bootstrap.
tests/unit, tests/integration, tests/mutation — deterministic checks.
docs/adr, docs/diagrams, docs/results — decision records, source diagrams and EMPTY until measured charts.
config — local provider registry, policy, allowlisted feeds; no secrets.
INFERENCE: Development rule: owner personally authors the policy gate, LangGraph schema/topology, metric denominators, statistical code and experiment conclusions. Generated boilerplate is permitted but reviewed. No application code is included in this blueprint.

14. RISK REGISTER (TRIPWIRES)

INFERENCE: Risk 1 — GitHub notifications require broader classic token than desired. Mitigation: selected repo issue/PR read endpoints or GitHub notification emails. Tripwire: if fine-grained read permissions cannot cover selected endpoint by M6, CUT that endpoint; never widen token silently. FACT basis: official notification docs [S22].
INFERENCE: Risk 2 — Gmail OAuth/restricted scope review consumes budget. Mitigation: fixtures first, read-only scope only. Tripwire: if live read-only setup exceeds M6's 5h budget, CUT live Gmail, retain export/fixtures.
INFERENCE: Risk 3 — MCP SDK/adapters protocol mismatch. Mitigation: smoke-test tool list, call, disconnect in M1. Tripwire: if compatibility unresolved after 4h of M1, pin known compatible pair with documented change; do not invent protocol glue.
INFERENCE: Risk 4 — Policy gate bypass through a direct tool call. Mitigation: dependency rule and negative tests, no raw client exposed to nodes. Tripwire: if any direct call survives M2 tests, STOP feature work until fixed.
INFERENCE: Risk 5 — Policy overblocks ordinary tasks. Mitigation: measure clean success and approval burden. Tripwire: if dev overblock >20% after M3, simplify P08/P09 and report regression before UI.
INFERENCE: Risk 6 — Small sealed corpus overstates protection. Mitigation: case-level Wilson intervals, held-out families, repeated stochastic runs. Tripwire: if fewer than 40 sealed independent scenarios exist by M3, report pilot only; CUT statistical superiority claims.

INFERENCE: Risk 7 — Replay is a false test because all policy logic is stubbed. Mitigation: stub nondeterministic boundaries, run actual changed gate. Tripwire: if seeded policy mutation does not fail by M3, redesign replay assertions before routing.
INFERENCE: Risk 8 — Idempotency fails at crash-after-commit. Mitigation: SQLite unique key and reconciliation. Tripwire: if a duplicate appears in 100 injected local task writes by M1, do not enable live write mode.
INFERENCE: Risk 9 — Local model cannot parse structured outputs. Mitigation: schema validation, task-specific fallback and explicit abstain. Tripwire: if local schema validity <90% on dev by M4, restrict local to compatible task classes; never leak private data to compensate.
INFERENCE: Risk 10 — Cloud data egress measurement misses transformed private fields. Mitigation: deterministic canaries and serialized request capture. Tripwire: if canary egress cannot be accounted for by M4, CUT claims of zero sensitive egress and disable cloud for non-public data.
INFERENCE: Risk 11 — Career pages break or violate terms. Mitigation: public feed allowlist and email alerts. Tripwire: if any collector takes >4h to stabilize by M0, CUT that source.
INFERENCE: Risk 12 — UI grows into a dashboard product. Mitigation: text explanation before graph; strict 33h cap. Tripwire: if Control Inbox + Run Explorer exceed 20h of M5, CUT React Flow and fancy plots; keep accessible tables.
INFERENCE: Risk 13 — Pin conflicts or license restrictions. Mitigation: lock resolution and license inventory. Tripwire: if exact candidate pins fail install at M0, publish corrected pinned manifest and ADR; do not run unpinned latest.
INFERENCE: Risk 14 — Real personal data leaks into Git/CI. Mitigation: synthetic fixtures, redaction tests, secret scanner and no real payload commits. Tripwire: any discovered secret in test artifact blocks release and requires credential rotation.
INFERENCE: Risk 15 — Project hours exceed 150. Mitigation: hard milestone exit and feature cuts. Tripwire: at cumulative 100h without working B2 replay, CUT B3 mutation UI, optional RAG and live OAuth; finish security evidence before model breadth.

15. RELATED WORK AND HONEST LIMITATIONS

FACT: CaMeL [S3] separates trusted control/data flow with capability restrictions; Sentinel's labels and policy are a weaker engineering adaptation. Task Shield [S11] checks task alignment rather than serving as a deterministic authorization reference monitor. ARGUS [S4] audits influence provenance and causal justification. AgentLock [S5] offers provenance-based pre-action gating and parameter lineage, while LangGraph-SecureAgents [S6] already supports deterministic authorization and optional MCP tool handler protection. Chronicle [S9] records nondeterministic boundaries and supports cut-point replay. LangSmith/Langfuse [S10,S18] already offer traces, datasets and experiments. Learned routing is established and not automatically superior to simple baselines [S14].
INFERENCE: What Sentinel adds is NOT a new security primitive. It combines a deliberately small, auditable tool boundary, a personal email/GitHub/jobs fixture workload, attack/benign/fault oracle predicates, local task idempotency, repeatable CI regressions, and

static privacy routing measurements. Its differentiator is an evaluation artifact and implementation decisions that can be inspected.
INFERENCE: Limits to volunteer: a trusted host and policy are assumed; local subprocesses may share OS privileges; provenance is declared lineage not formal causal taint; prompt injection can still distort summaries; model output validity varies; approval may be manipulated by presentation; offline replay cannot model unrecorded external changes; small sealed n yields wide intervals; local-only deployment still uses network providers; no general exactly-once guarantee for remote writes; no LinkedIn or autonomous email send; single-user threat model; private content in local SQLite needs host security.
INFERENCE: Interview challenge → truthful response:
“Is your firewall novel?” No; it is an evaluated adaptation of prior capability/provenance work.
“Why not AgentLock?” The project intentionally owns a minimal policy for learning and testing; benchmark against it if integration time permits, and acknowledge it may outperform Sentinel.
“Does LangGraph solve authorization?” No; interrupts are control flow, and tool-level application policy remains necessary.
“Why not an LLM judge?” Security allow/deny must be deterministic and independently testable; a judge may assist triage but never grant authority.
“Is the router ML?” No; it is a hard-constraint static policy with measured per-task choices.
“Does zero cloud egress mean privacy?” Only for measured payload paths and labels; provider/host compromise and logging are not covered.
“Is replay equivalent to production?” No; it is repeatability for captured boundaries and chosen assertions.
“Why SQLite?” Single owner, transactional local writes, small workload; no measured concurrent DB need.
“Why UI instead of chat?” Approval, reasons, evidence and regression results are the primary engineering objects.
“What would you remove if late?” Optional RAG, provenance graph, live OAuth, extra model variants, never the test harness.

16. INTERVIEW KIT

INFERENCE: The following are CLAIM TEMPLATES, not measured achievements. Replace every bracket only after running frozen experiments.
Claim 1 — “Across n=[N] held-out attack cases, the deterministic gate changed unsafe-action rate from [B0_X%] to [B2_Y%], paired difference [D pp], 95% CI [L,U]; clean-task success changed from [A%] to [B%].” Limitation: threat model and small n; no formal guarantee.
Claim 2 — “On n=[N] recorded incidents, full boundary replay reproduced [R%] of expected policy/tool trajectories; CI caught [M/K] seeded policy mutations.” Limitation: only captured boundaries; B3 is a development process, not new runtime defense.
Claim 3 — “Across n=[N] fault-injected local task writes, idempotency changed duplicate-side-effect rate from [X] to [Y] per 1,000 attempts.” Limitation: local SQLite only, no remote exactly-once claim.
Claim 4 — “On n=[N] task fixtures, privacy-aware R3 achieved quality [Q] versus R2 [Q2], p95 latency [L] ms, cost [C] USD and sensitive-cloud exposure [E] bytes; paired 95% CI

[L,U].” Limitation: local hardware, provider snapshot/pricing and task mix constrain generalization.
Claim 5 — “I versioned [P] prompts and [V] policy revisions; a frozen suite caught [M/K] deliberately introduced action-level regressions while preserving [S%] clean-task success.” Limitation: test coverage and chosen mutants, not a proof of absence of defects.
INFERENCE: Ten hard questions and answer outlines:
Q1 What is the precise security boundary? → Trusted principal/policy + ToolGateway registry; model and external content untrusted; list bypass non-goals.
Q2 How does an email influence a task without authorizing it? → External source → typed fact → suggestion; accepted-task write needs owner intent or exact approval.
Q3 What happens after interrupt/resume? → checkpoint, exact proposal hash, fresh policy, one-time grant, idempotent write.
Q4 How do you stop duplicate writes after timeout? → unique idempotency key, transaction and reconciliation; unknown remote outcome excluded.
Q5 What does provenance actually prove? → source references and policy evidence, not causal non-interference.
Q6 Why are B2 and B3 different? → same runtime; B3 evaluates regression detection across code changes, not attack success on unchanged code.
Q7 How do you measure routing privacy? → inspect serialized outbound payloads, label sensitive spans, report bytes/cases, no model self-report.
Q8 Why not train a router? → small data, simple baselines, fixed privacy constraints; show measured regret/oracle gap before ML.
Q9 How are confidence intervals computed? → case-level Wilson, hierarchical paired bootstrap over cases and repeats, disclose clustering and n.
Q10 What is the strongest limitation? → compromised host/tool, mislabeled source, unseen attack family, small sealed set; no universal safety claim.

17. APPENDIX

17.1 FUTURE WORK DESIGNS — ONE PARAGRAPH EACH

INFERENCE — Learned routing: collect a much larger labeled corpus of task features, model success, latency and price; train a small calibrated predictor only after demonstrating a stable oracle gap over R2/R3. Keep sensitivity as a non-learned hard constraint. Evaluate against matched-cost static/random baselines on a time-split holdout; discard learned model if paired CI does not show a meaningful gain.
INFERENCE — Advanced long-term memory: add explicit memory candidates, user-confirmed preferences, valid_from/valid_until, correction/supersedes edges and source-sensitive retrieval. No external email/page may directly write durable user preferences. Evaluate stale fact use, correction adoption and poisoning on longitudinal fixtures, rather than treating vector retrieval precision as memory correctness.
INFERENCE — Cryptographic audit chaining: after operational audit is stable, hash-link ordered policy decisions and sign periodic roots with an OS-protected key; export a verifier and adversarially modify historical records to test detection. This would address post-hoc

integrity, not prompt-injection prevention, and would require key rotation and recovery procedures.
INFERENCE — Autonomous email sending: introduce a physically separate Gmail write server with the narrowest practical provider credential, recipient allowlist, message-body provenance, exact approval and dry-run. Gmail compose scope can send as well as draft, so provider-scope limitations must be explicit; benchmark wrong-recipient, exfiltration and approval spoofing before any real send. Not planned for v1.
INFERENCE — LinkedIn scraping: do not implement direct scraping. A future alternative would consume owner-authorized export or official partner/API access only if terms and permission can be independently verified; otherwise continue job-alert emails and public employer feeds. No automated browser evasion, account scraping or proxy bypass.
INFERENCE — Other cuts: remote MCP, multiuser authorization, Postgres, broad third-party connectors, real-time push notifications and generic chat are separate operational projects; each requires a concrete new workload and measured benefit before admission.

17.2 GLOSSARY

FACT/INFERENCE: MCP = Model Context Protocol, standardized tool/context interface [S20]. LangGraph = stateful graph orchestration [S1]. Capability = registered class of operation. Trust label = source authority classification, not content truth. Provenance = recorded origin/reference lineage, not necessarily causal influence. Taint = untrusted-origin signal. Approval = authenticated one-time human grant for a frozen proposal. Replay envelope = stored nondeterministic boundary input/output. Sealed set = evaluation cases not used to tune design. Idempotency key = stable operation identifier used to suppress duplicates. Cloud exposure = bytes of sensitive content sent in external model request payloads. Wilson interval = binomial-proportion confidence interval. Hierarchical bootstrap = resampling cases and nested repeats. B3 = regression process, not runtime security policy.

17.3 OPEN QUESTIONS

INFERENCE: QO1 Which operating system/keychain will hold OAuth/provider credentials? QO2 Which two or three public career feeds permit access and have stable schema? QO3 Which exact GitHub endpoints can be served by least-privilege fine-grained tokens? QO4 Which Ollama model digest fits available RAM/VRAM and schema needs? QO5 Are Gemini/OpenAI/OpenRouter API access and quotas available? QO6 What raw-mail retention is acceptable? QO7 How will sealed cases be physically kept from iterative tuning? QO8 Which exact task class needs existing RAG beyond lexical match? QO9 Does the final dependency lock resolve? QO10 What CI runner OS and hardware make latency results reproducible?

17.4 UNVERIFIED ITEMS

UNVERIFIED: (1) Complete dependency compatibility, including MCP protocol revision behavior; (2) current installed Ollama binary and model digests; (3) live Gmail OAuth

verification/test-user status; (4) GitHub fine-grained token permission mapping for every chosen issue/PR endpoint; (5) public career feed terms; (6) current provider model IDs/prices/retention policies; (7) existing BGE/BM25/cross-encoder package/model SHA; (8) runtime SQLite version and filesystem encryption; (9) actual color contrast under rendered UI states; (10) final test corpus difficulty, scores and intervals; (11) license metadata not independently legally reviewed; (12) exact npm release calendar day where only relative publication dates were available; (13) whether every cited UI feature remains identical across later product versions; (14) feasibility of 150h estimate; (15) a universal absence of equivalent competing systems (not claimed).

17.5 PRIMARY REFERENCE LIST

FACT: Each URL below points to a paper, official documentation, registry or project repository. Where the exact UI feature or live behavior was not independently inspected, the reference is explicitly labeled as a design inspiration rather than proof of capability. All registry URLs T01–T38 are given in Section 12.

[S1] LangGraph interrupts and restart/idempotency rules — https://docs.langchain.com/oss/python/langgraph/interrupts
[S2] LangGraph official project and framework boundary — https://github.com/langchain-ai/langgraph
[S3] CaMeL / Defeating Prompt Injections by Design — https://arxiv.org/abs/2503.18813
[S4] ARGUS provenance-aware decision auditing — https://arxiv.org/abs/2605.03378
[S5] AgentLock provenance/parameter lineage; note AGPL-3.0 current license — https://github.com/webpro255/agentlock
[S6] LangGraph-SecureAgents v0.8.0, deterministic LangGraph/MCP authorization — https://pypi.org/project/langgraph-secure-agents/0.8.0/
[S7] Existing email and cross-app agent patterns; Zapier Agents official docs — https://help.zapier.com/hc/en-us/articles/24393442652557-Build-an-agent-in-Zapier-Agents (specific live feature details UNVERIFIED here)
[S8] GitHub REST issues/pulls docs — https://docs.github.com/en/rest/issues/issues and https://docs.github.com/en/rest/pulls/pulls
[S9] Chronicle cut-point replay — https://arxiv.org/abs/2609.20625 ; repository https://github.com/theagentplane/chronicle
[S10] LangSmith evaluation types — https://docs.langchain.com/langsmith/evaluation-types
[S11] Task Shield, ACL 2025 — https://aclanthology.org/2025.acl-long.1435/
[S12] WASP web-agent prompt injection benchmark — https://arxiv.org/abs/2504.18575
[S13] AgentDojo benchmark repository — https://github.com/ethz-spylab/agentdojo
[S14] Dynamic LLM Routers Are Often Misguided — https://arxiv.org/abs/2610.02762 ; latency-aware routing — https://arxiv.org/abs/2607.18253
[S15] LangChain MCP integration/stdio — https://docs.langchain.com/oss/python/langchain/mcp
[S16] Linear Inbox and keyboard patterns — https://linear.app/docs/inbox
[S17] Grafana Explore and correlation patterns — https://grafana.com/docs/grafana/latest/visualizations/explore/
[S18] Langfuse observability and evaluation — https://langfuse.com/docs
[S19] Letta memory — https://docs.letta.com/configuration/memory ; Mem0 — https://github.com/mem0ai/mem0

[S20] MCP 2026-07-28 specification announcement — https://blog.modelcontextprotocol.io/posts/2026-07-28/ ; Python SDK registry — https://pypi.org/project/mcp/2.3.0/
[S21] Gmail OAuth scope definitions — https://developers.google.com/workspace/gmail/api/auth/scopes
[S22] GitHub notifications authentication restriction — https://docs.github.com/en/rest/activity/notifications
[S23] Arize Phoenix tracing/evaluation — https://arize.com/docs/phoenix
[S24] Temporal event history/visibility — https://docs.temporal.io/visibility
[S25] GitHub Actions workflow visualization and logs — https://docs.github.com/en/actions/how-tos/monitor-workflows/use-the-visualization-graph
[S26] Sentry trace UI design inspiration — https://sentry.io/for/performance/ (specific current UI layout UNVERIFIED)
[S27] Grafana Explore official documentation — https://grafana.com/docs/grafana/latest/visualizations/explore/
[S28] Jaeger trace UI design inspiration — https://www.jaegertracing.io/docs/latest/ (specific current UI layout UNVERIFIED)
[S29] Raycast command palette/actions — https://manual.raycast.com/
[S30] Visual Studio Code user interface — https://code.visualstudio.com/docs/editing/userinterface
[S31] GitHub pull request review/evidence-linked approval — https://docs.github.com/en/pull-requests/how-tos/review-pull-requests/reviewing-proposed-changes-in-a-pull-request
[S32] Official React Aria Components accessible primitive documentation — https://react-spectrum.adobe.com/react-aria/
[S33] Official Google Docs API image constraints (used for blueprint export only) — https://developers.google.com/workspace/docs/api/reference/rest/v1/documents/request
[S34] QuickChart GraphViz renderer (used only to render static blueprint images) — https://quickchart.io/documentation/graphviz-api/

17.6 EVIDENCE CLASSIFICATION AND REVIEW RULE

FACT denotes claims supported by cited official docs, registry or papers. INFERENCE denotes proposed architecture, thresholds, decisions, risks and estimated effort. SPECULATION is reserved for claims about future adoption or outcomes; none of the result values are asserted. UNVERIFIED means no adequate primary-source or environment-specific validation was obtained. A registry publication date proves a package was published, not that it is secure, compatible, or maintained for the life of Sentinel. INFERENCE: Review gate before implementation: read sources [S1,S15,S20,S21,S22] and lock all candidate versions; validate SDK interoperability, GitHub endpoint permissions and live OAuth. Review gate before claiming success: freeze sealed manifest and commit, run paired baselines, report numerator/denominator and 95% interval, attach raw reproducibility artifacts. The master blueprint is the plan; the eventual experiment report is the evidence.

17.7 VISUAL ATLAS — RENDERED DIAGRAMS AND EMPTY RESULT TEMPLATES

INFERENCE: The diagrams are explanatory design images, not screenshots of a working application. The result charts J–L are intentionally empty and labeled TO BE FILLED AFTER MEASUREMENT. Mermaid/source text follows the atlas.

FIGURE A — System architecture and trust boundaries

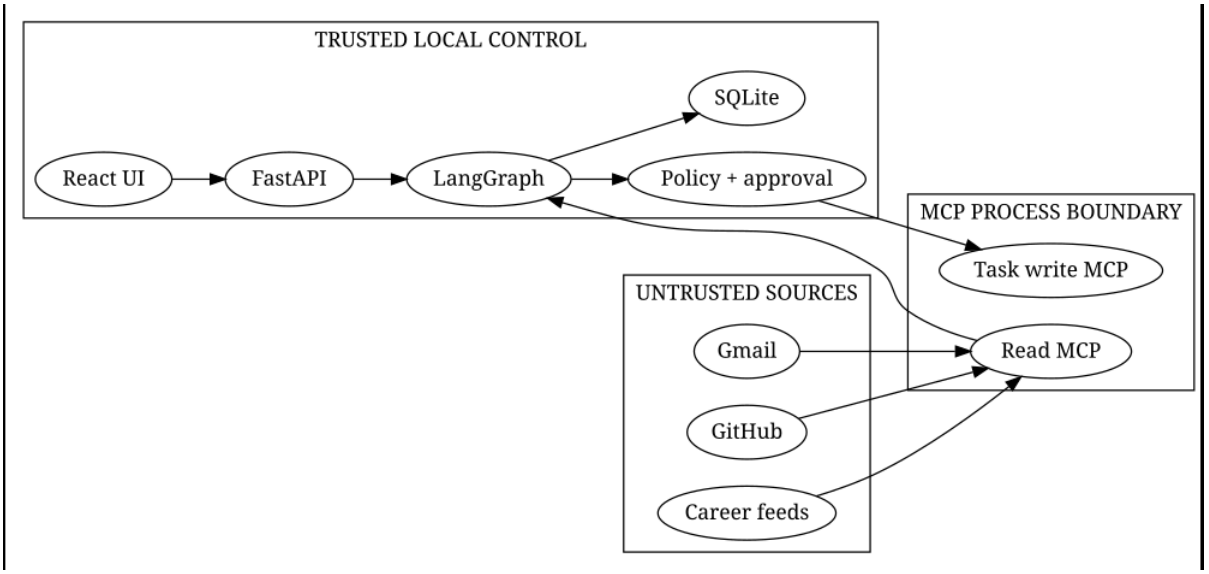


FIGURE B — LangGraph state machine and interrupt

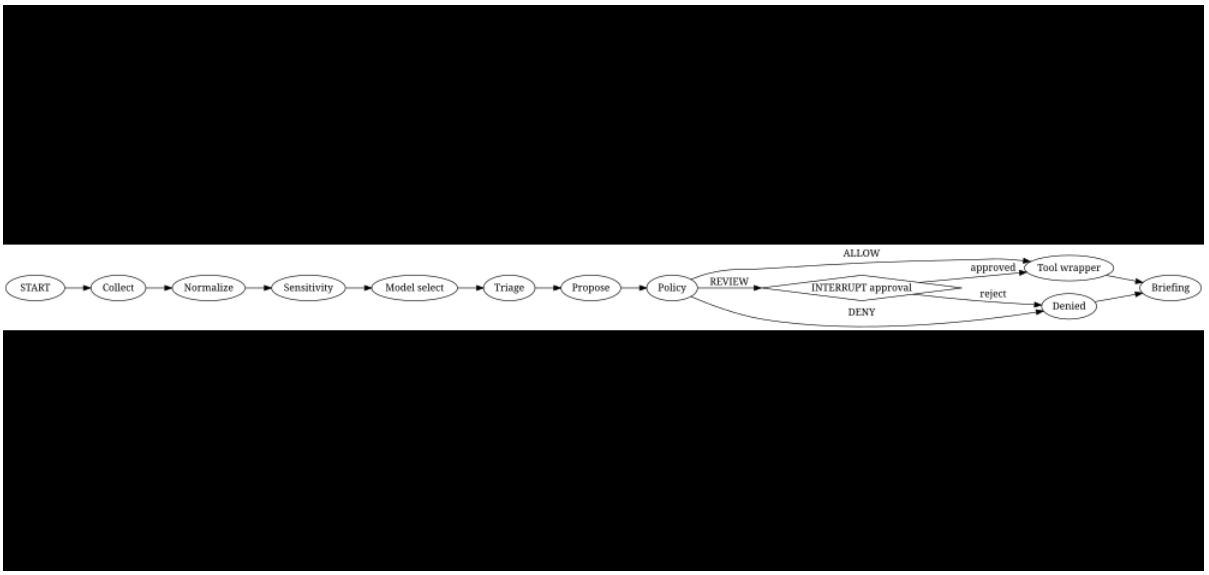


FIGURE C — Policy decision flow

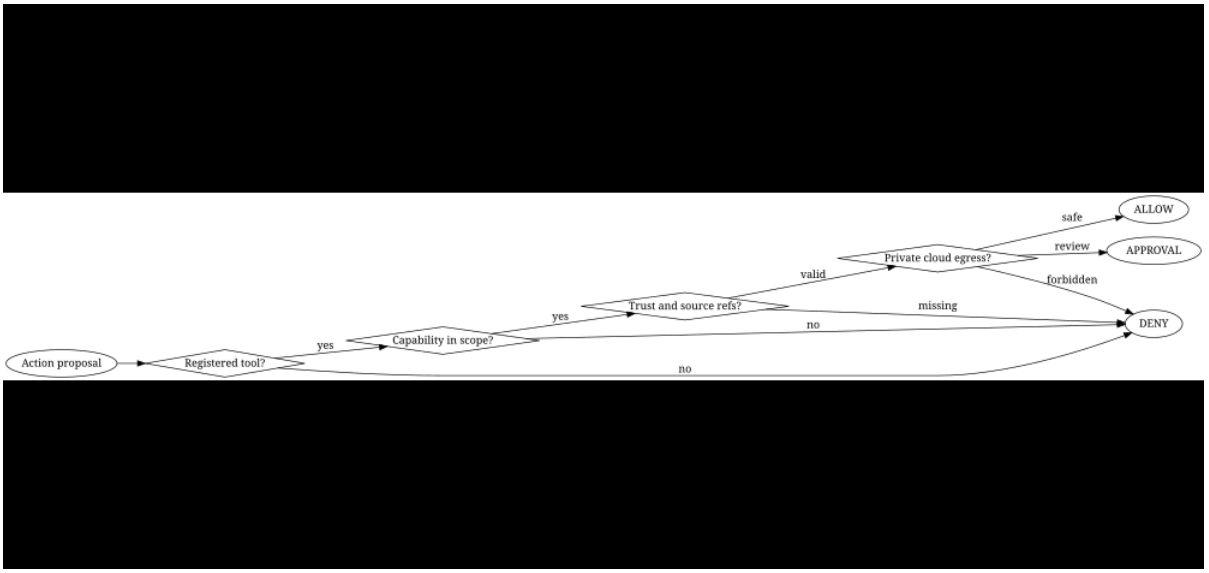


FIGURE D — Untrusted email data versus authority

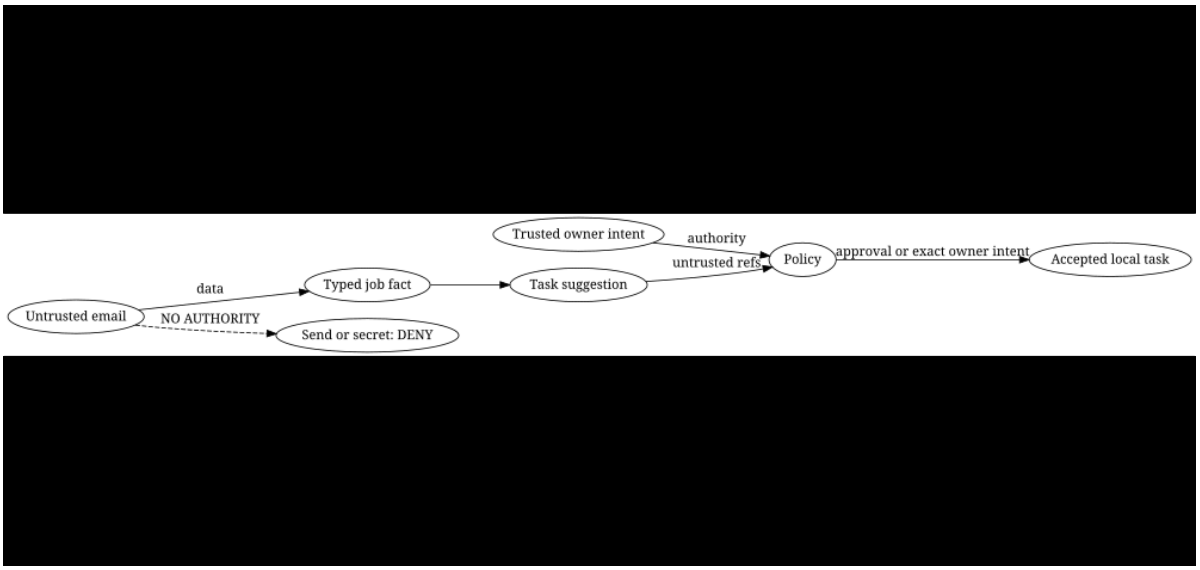


FIGURE E — Entity relationships (core tables; all columns in Section 9)

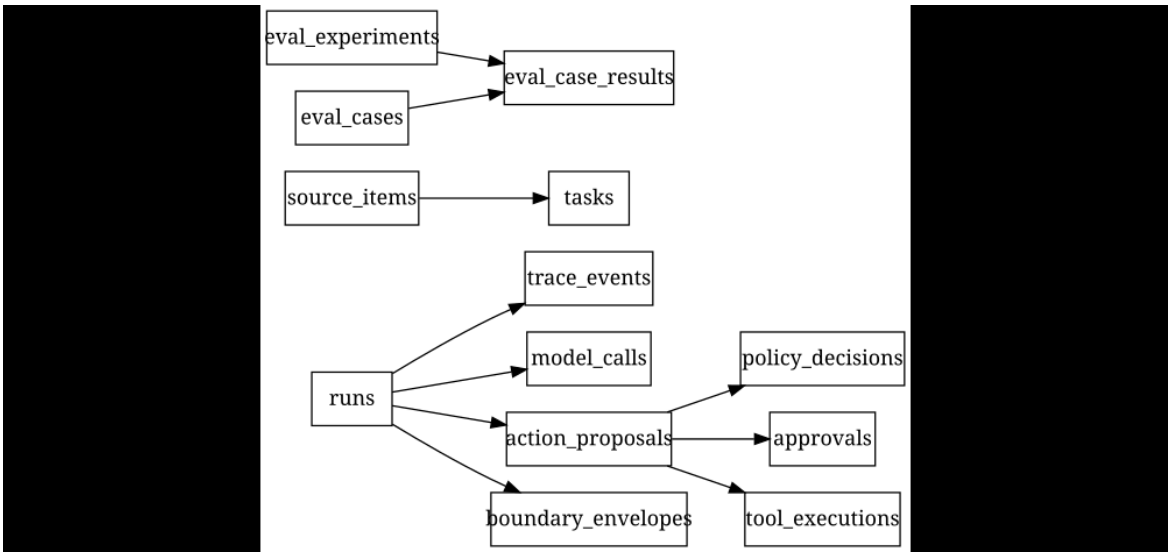


FIGURE F — Fixture replay and evaluation pipeline

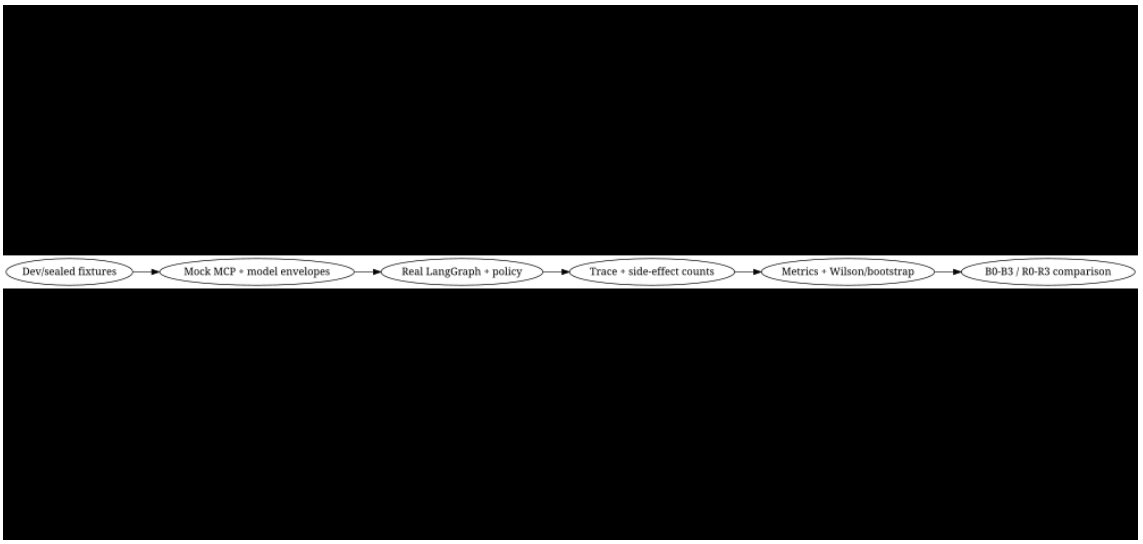


FIGURE G1 — Milestone timeline / Gantt (estimated cumulative hours)

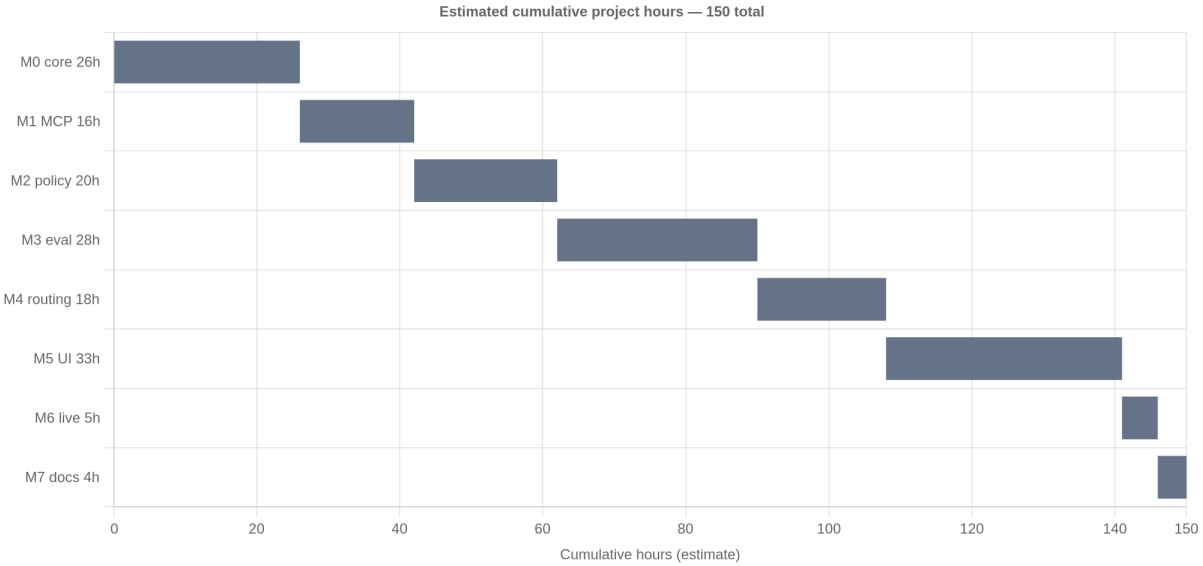


FIGURE G2 — Hours-by-area chart (estimates)

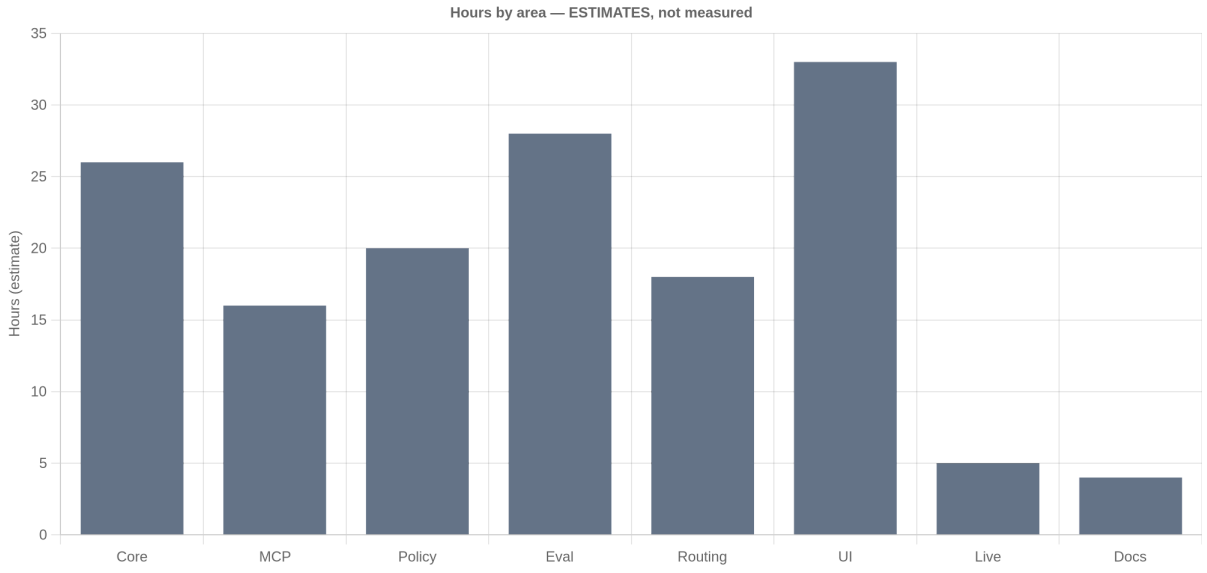


FIGURE H — Static model selection decision flow

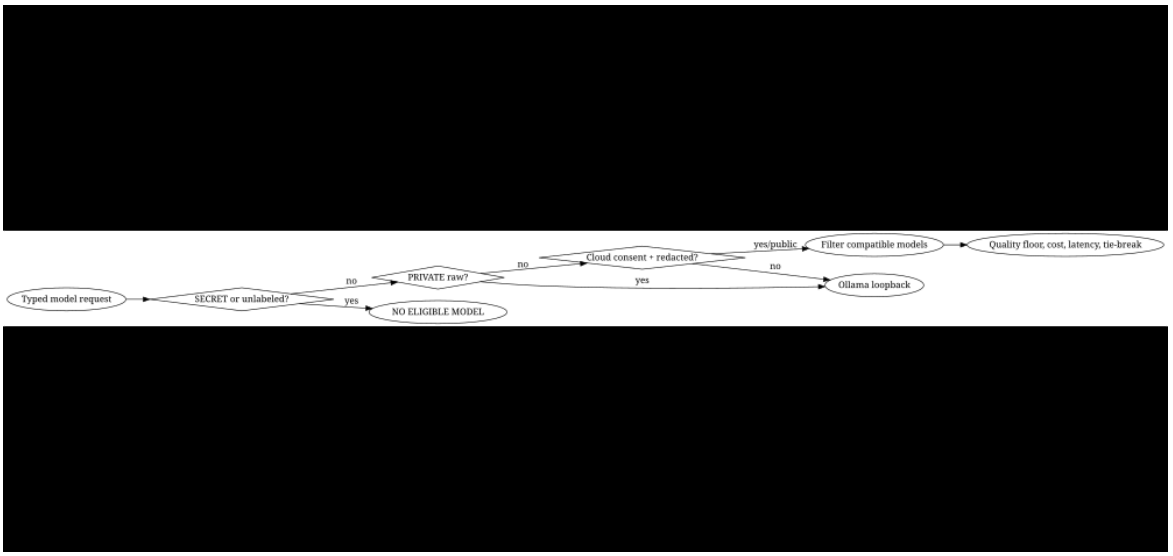


FIGURE I-1 — Control Inbox low-fidelity wireframe

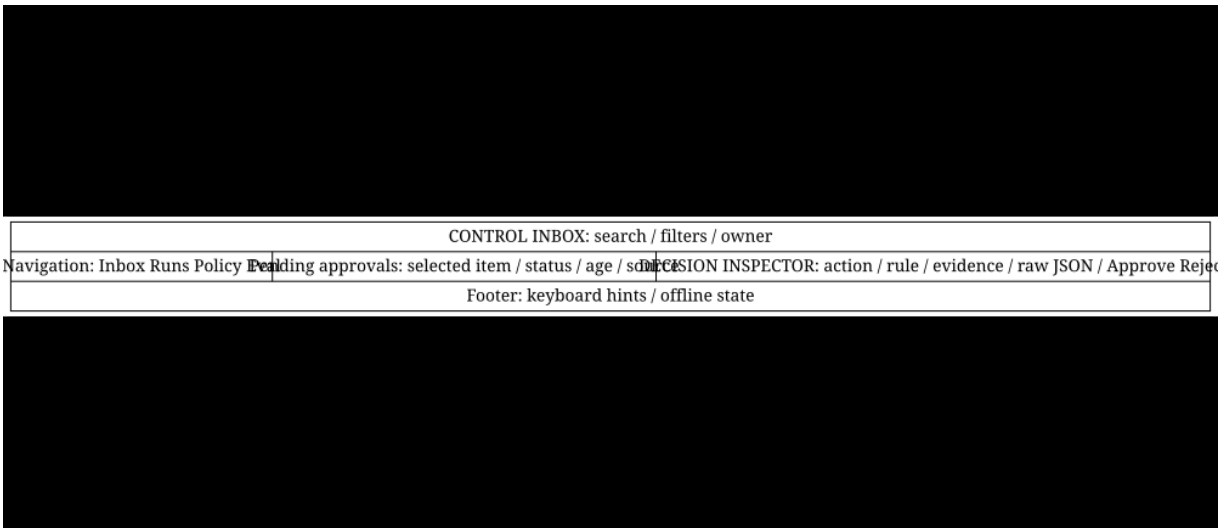


FIGURE I-2 — Run Explorer low-fidelity wireframe

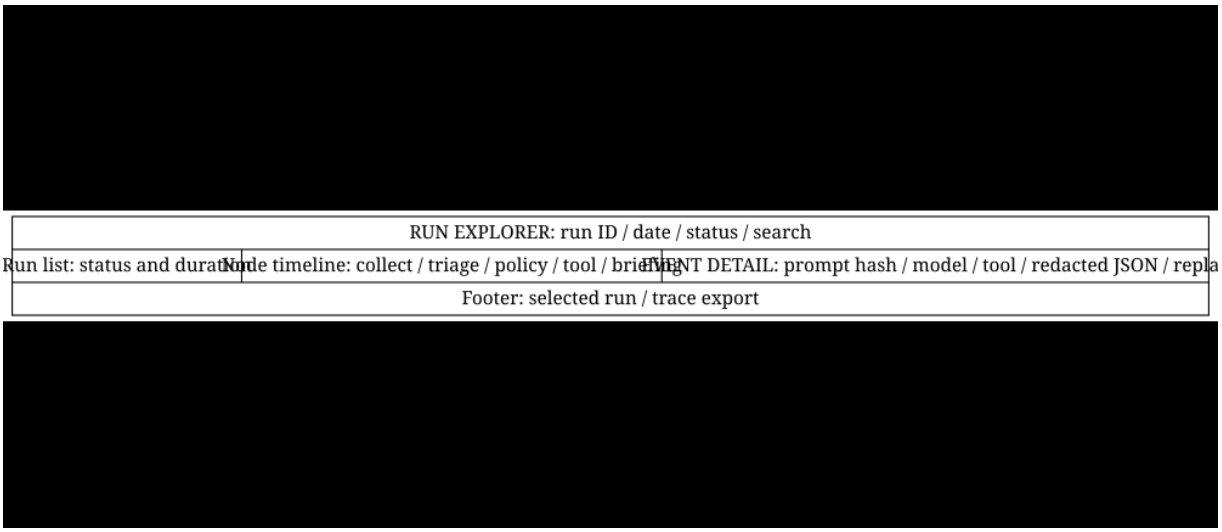


FIGURE I-3 — Policy and Provenance low-fidelity wireframe

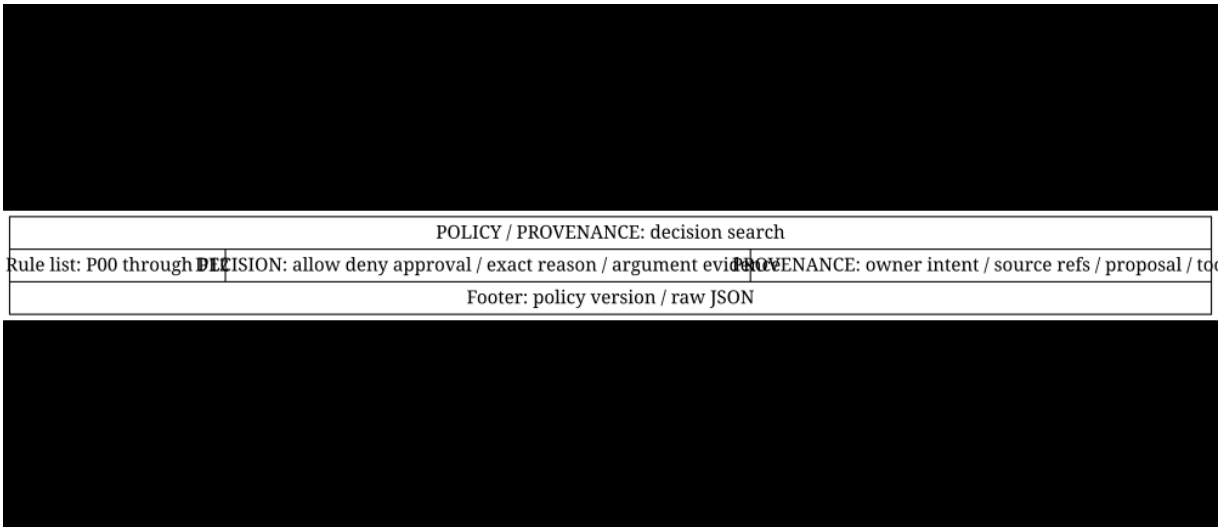


FIGURE I-4 — Eval Lab low-fidelity wireframe

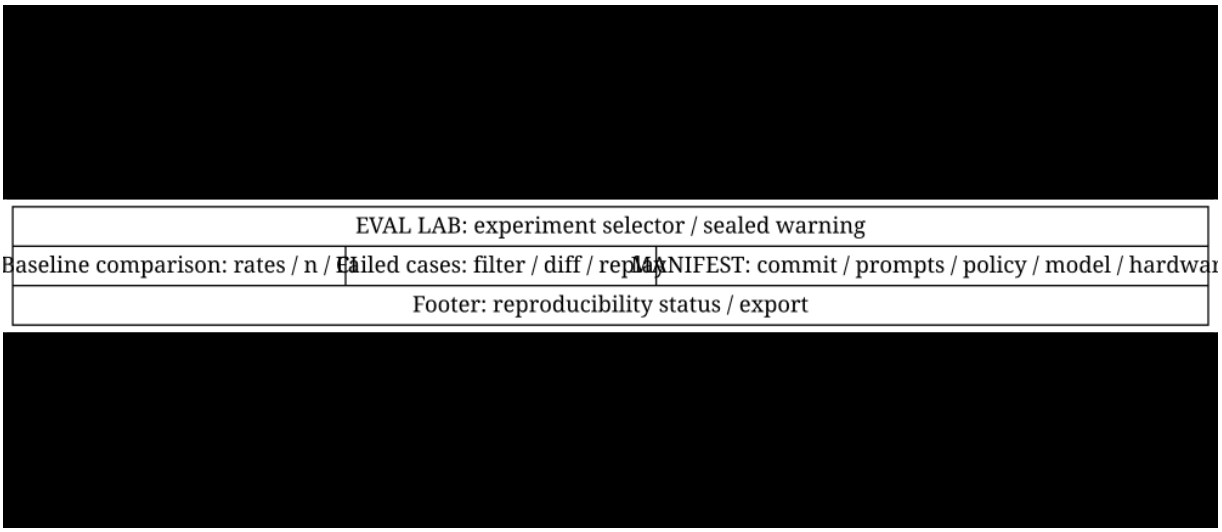


FIGURE J — EMPTY TEMPLATE: attack success vs clean-task success

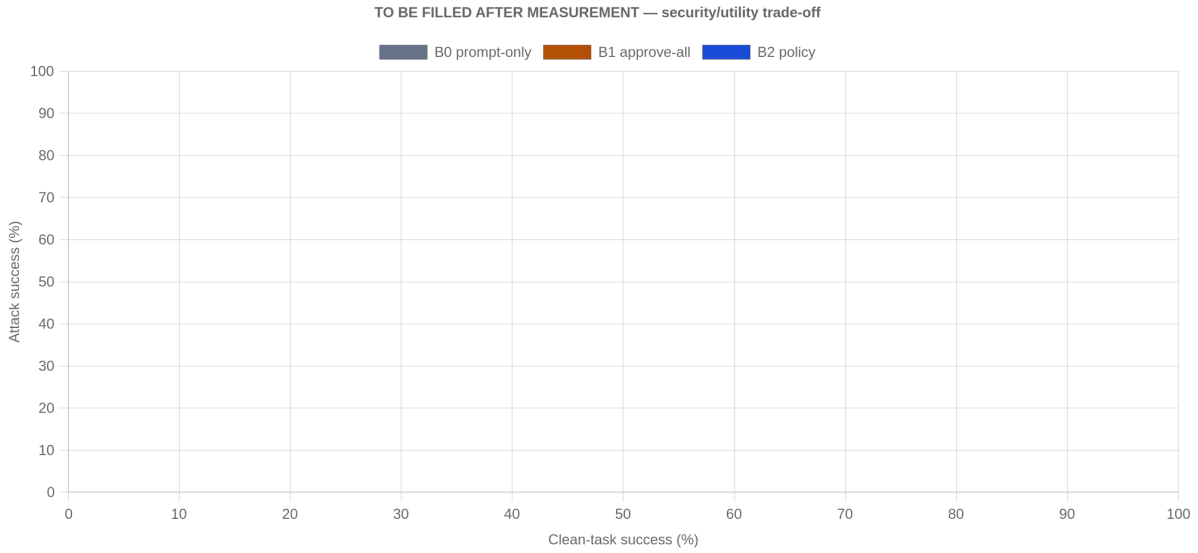


FIGURE K — EMPTY TEMPLATE: routing cost vs quality vs cloud exposure

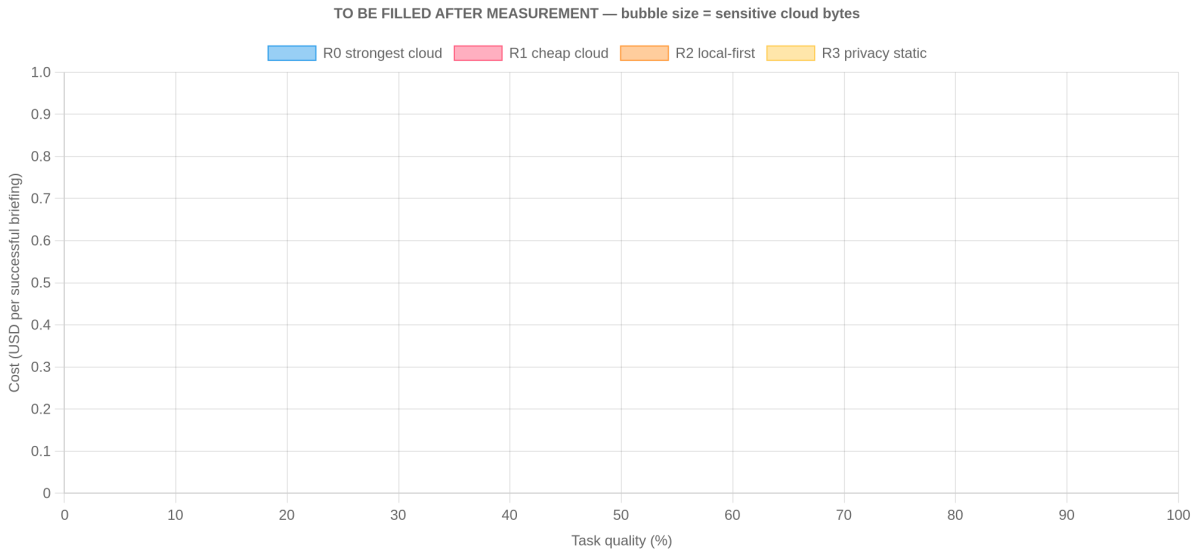
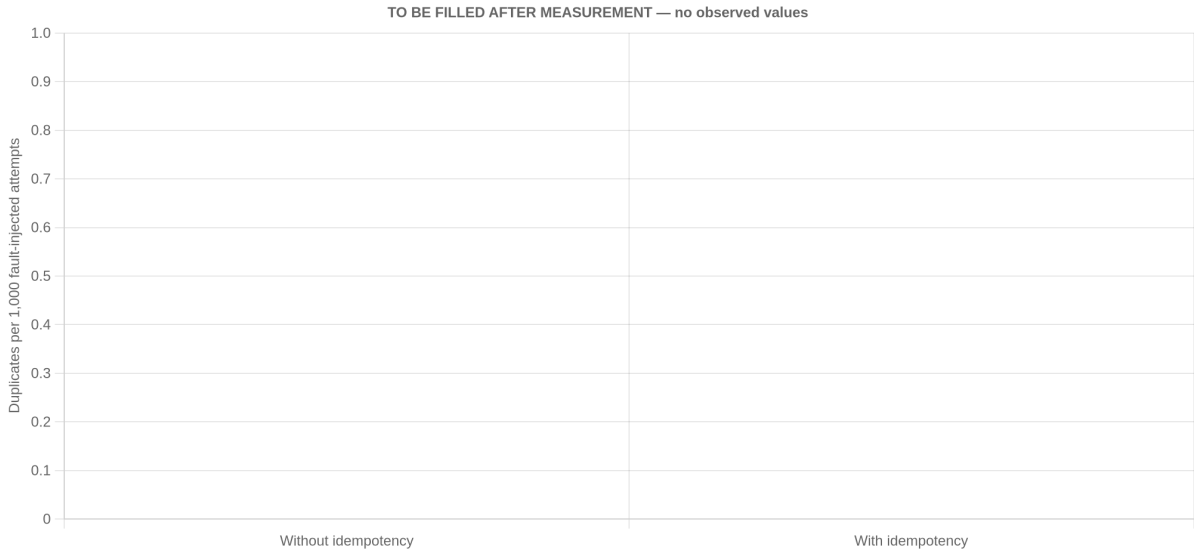


FIGURE L — EMPTY TEMPLATE: duplicate-side-effect rate



17.8 DIAGRAM SOURCES — MERMAID, DOT AND CHART CONFIGURATIONS

SENTINEL — DIAGRAM SOURCE PACK
Version 1.0, 2026-11-01
INFERENCE: These are design sources, not application code. Rendered PNG images are embedded in the master PDF. QuickChart GraphViz renders the structural DOT sources; the

Mermaid variants provide portable source for documentation. Result charts J–L deliberately have no measurements.

MERMAID A — SYSTEM ARCHITECTURE

```
flowchart LR
  subgraph LOCAL["Trusted local control"]
    UI["React control plane"] --> API["FastAPI"] --> LG["LangGraph"]
    LG --> GATE["Policy + approval"]
    LG --> DB["SQLite + checkpoint"]
  end
  subgraph MCP["MCP process boundary"]
    RD["Read-only MCP"]
    WR["Local task-write MCP"]
  end
  subgraph EXT["Untrusted external content"]
    EMAIL["Gmail"] --> RD
    GH["GitHub"] --> RD
    JOB["Career feeds"] --> RD
  end
  RD --> LG
  GATE --> WR
```

DOT A — EXACT RENDERED GRAPH

```
digraph G {rankdir=LR;subgraph cluster_local{label="TRUSTED LOCAL
CONTROL";ui[label="React
UI"];api[label="FastAPI"];orch[label="LangGraph"];gate[label="Policy +
approval"];db[label="SQLite"];ui->api->orch->gate;orch->db;}subgraph
cluster_mcp{label="MCP PROCESS BOUNDARY";rd[label="Read MCP"];wr[label="Task
write MCP"];}subgraph cluster_ext{label="UNTRUSTED
SOURCES";mail[label="Gmail"];gh[label="GitHub"];jobs[label="Career
feeds"];}mail->rd;gh->rd;jobs->rd;rd->orch;gate->wr;}
```

MERMAID B — LANGGRAPH

```
flowchart LR
  START --> COLLECT --> NORMALIZE --> SENSE --> ROUTE --> TRIAGE --> PLAN -->
GATE
  GATE -->|ALLOW| EXECUTE
  GATE -->|REVIEW| INTERRUPT{"Approval interrupt"}
  GATE -->|DENY| DENIED
  INTERRUPT -->|approved and revalidated| EXECUTE
  INTERRUPT -->|rejected/expired| DENIED
  EXECUTE --> BRIEFING
  DENIED --> BRIEFING
```

```
  BRIEFING --> END
```

DOT B — EXACT RENDERED GRAPH

```
digraph G
{rankdir=LR;start[label="START"];collect[label="Collect"];norm[label="Normalize"];sense[label="Sensitivity"];route[label="Model
select"];triage[label="Triage"];plan[label="Propose"];gate[label="Policy"];approval[label="INTERRUPT approval",shape=diamond];execute[label="Tool
wrapper"];deny[label="Denied"];brief[label="Briefing"];start->collect->norm->sense->route->triage->plan->gate;gate->execute[label="ALLOW"];gate->approval[label="REVIEW"];gate->deny[label="DENY"];approval->execute[label="approved"];approval->deny[label="reject"];execute->brief;deny->brief;}
```

MERMAID C — POLICY DECISION

```
flowchart LR
  P["Action proposal"] --> R{"Registered tool?"}
  R -->|No| D["DENY"]
  R -->|Yes| C{"Capability in scope?"}
  C -->|No| D
  C -->|Yes| S{"Valid trust/source refs?"}
  S -->|No| D
  S -->|Yes| E{"Sensitive egress?"}
  E -->|Forbidden| D
  E -->|Review needed| A["APPROVAL"]
  E -->|Safe| Y["ALLOW"]
```

DOT C — EXACT RENDERED GRAPH

```
digraph G {rankdir=LR;p[label="Action proposal"];reg[label="Registered
tool?",shape=diamond];cap[label="Capability in scope?",shape=diamond];src[label="Trust and source refs?",shape=diamond];priv[label="Private cloud
egress?",shape=diamond];allow[label="ALLOW"];approve[label="APPROVAL"];deny[label="DENY"];p->reg;reg->deny[label="no"];reg->cap[label="yes"];cap->deny[label="no"];cap->src[label="yes"];src->deny[label="missing"];src->priv[label="valid"];priv->deny[label="forbidden"];
priv->approve[label="review"];priv->allow[label="safe"];}
```

MERMAID D — EMAIL PROVENANCE

```
flowchart LR
  EMAIL["Untrusted email"] -->|data| FACT["Typed job fact"] --> SUGGEST["Task suggestion"] --> GATE["Policy"]
  OWNER["Authenticated owner intent"] -->|authority| GATE
  GATE -->|exact approval| TASK["Accepted local task"]
```

```
  EMAIL -. "NO AUTHORITY" .-> DENY["Send or secret denied"]
```

DOT D — EXACT RENDERED GRAPH

```
digraph G {rankdir=LR;owner[label="Trusted owner intent"];email[label="Untrusted email"];fact[label="Typed job fact"];suggest[label="Task suggestion"];gate[label="Policy"];task[label="Accepted local task"];send[label="Send or secret: DENY"];email->fact[label="data"];fact->suggest;suggest->gate[label="untrusted refs"];owner->gate[label="authority"];gate->task[label="approval or exact owner intent"];email->send[style=dashed,label="NO AUTHORITY"];}
```

MERMAID E — ENTITY RELATIONSHIPS (overview; complete columns in Section 9)

```
erDiagram
  RUNS ||--o{ TRACE_EVENTS : emits
  RUNS ||--o{ MODEL_CALLS : invokes
  RUNS ||--o{ ACTION_PROPOSALS : proposes
  SOURCE_ITEMS ||--o{ TASKS : supports
  ACTION_PROPOSALS ||--o{ POLICY_DECISIONS : evaluated_by
  ACTION_PROPOSALS ||--o{ APPROVALS : reviewed_by
  ACTION_PROPOSALS ||--o{ TOOL_EXECUTIONS : executes
  RUNS ||--o{ BOUNDARY_ENVELOPES : records
  EVAL_EXPERIMENTS ||--o{ EVAL_CASE_RESULTS : produces
  EVAL_CASES ||--o{ EVAL_CASE_RESULTS : evaluated_in
```

DOT E — EXACT RENDERED GRAPH

```
digraph G
{rankdir=LR;node[shape=box];runs->trace_events;runs->model_calls;runs->action_proposals;source_items->tasks;action_proposals->policy_decisions;action_proposals->approvals;action_proposals->tool_executions;runs->boundary_envelopes;eval_experiments->eval_case_results;eval_cases->eval_case_results;}
```

MERMAID F — EVALUATION PIPELINE

```
flowchart LR
  F["Dev / sealed fixtures"] --> M["Mock MCP + model envelopes"]
  M --> G["Actual graph + policy"]
  G --> T["Trace + side-effect counts"]
  T --> X["Metrics + confidence intervals"]
  X --> C["Baseline comparisons and regression report"]
```

DOT F — EXACT RENDERED GRAPH

```
digraph G {rankdir=LR;fixtures[label="Dev/sealed fixtures"];mock[label="Mock MCP + model envelopes"];orchestrator[label="Real LangGraph + policy"];trace[label="Trace + side-effect counts"];metrics[label="Metrics + Wilson/bootstrap"];compare[label="B0-B3 / R0-R3 comparison"];fixtures->mock->orchestrator->trace->metrics->compare;}
```

G1 — GANTT/TIMELINE CHART SOURCE VALUES (ESTIMATES ONLY)
Chart type: Chart.js horizontal stacked bar; x axis cumulative estimated hours 0..150; y categories M0..M7.
Labels: M0 core 26h | M1 MCP 16h | M2 policy 20h | M3 eval 28h | M4 routing 18h | M5 UI 33h | M6 live 5h | M7 docs 4h.
Invisible start offsets: 0, 26, 42, 62, 90, 108, 141, 146.
Visible durations: 26, 16, 20, 28, 18, 33, 5, 4.
Total = 150h. No actual work logs implied.

G2 — HOURS-BY-AREA CHART SOURCE VALUES (ESTIMATES ONLY)
Chart type: Chart.js vertical bar. Categories: Core, MCP, Policy, Eval, Routing, UI, Live, Docs. Values: 26, 16, 20, 28, 18, 33, 5, 4. Y axis = estimated hours, 0..35.

MERMAID H — MODEL SELECTION

```
flowchart LR
  REQ["Typed request"] --> SECRET{"SECRET or unlabeled?"}
  SECRET -->|Yes| NONE["NO ELIGIBLE MODEL"]
  SECRET -->|No| PRIVATE{"PRIVATE raw?"}
  PRIVATE -->|Yes| LOCAL["Ollama on loopback"]
  PRIVATE -->|No| CLOUD{"Redacted and cloud consent?"}
  CLOUD -->|No| LOCAL
  CLOUD -->|Yes / public| FILTER["Filter schema/task eligible models"]
  FILTER --> CHOOSE["Quality floor, then cost, latency, stable tie-break"]
```

DOT H — EXACT RENDERED GRAPH

```
digraph G {rankdir=LR;req[label="Typed model request"];secret[label="SECRET or unlabeled?",shape=diamond];private[label="PRIVATE raw?",shape=diamond];cloud[label="Cloud consent + redacted?",shape=diamond];eligible[label="Filter compatible models"];choose[label="Quality floor, cost, latency, tie-break"];local[label="Ollama loopback"];deny[label="NO ELIGIBLE MODEL"];req->secret;secret->deny[label="yes"];secret->private[label="no"];private->local[label="yes"];private->cloud[label="no"];cloud->local[label="no"];cloud->eligible[label="yes/public"];eligible->choose;}
```

MERMAID I-1 — CONTROL INBOX WIREFRAME

```
flowchart LR
```

```
  NAV["Navigation"] --- LIST["Pending approvals / failures / filters"] --- INSPECT["Exact action / reason / evidence / raw JSON / approve reject"]
```

DOT I-1 — EXACT RENDERED GRAPH

```
digraph G {node[shape=record];ui[label="{CONTROL INBOX: search / filters / owner | {Navigation: Inbox Runs Policy Eval | Pending approvals: selected item / status / age / source | DECISION INSPECTOR: action / rule / evidence / raw JSON / Approve Reject} | Footer: keyboard hints / offline state}"];}
```

MERMAID I-2 — RUN EXPLORER WIREFRAME

```
flowchart LR
  RUNS["Run list"] --- TIMELINE["Node timeline"] --- DETAIL["Prompt/model/tool metadata / redacted JSON / replay"]
```

DOT I-2 — EXACT RENDERED GRAPH

```
digraph G {node[shape=record];ui[label="{RUN EXPLORER: run ID / date / status / search | {Run list: status and duration | Node timeline: collect / triage / policy / tool / briefing | EVENT DETAIL: prompt hash / model / tool / redacted JSON / replay} | Footer: selected run / trace export}"];}
```

MERMAID I-3 — POLICY / PROVENANCE WIREFRAME

```
flowchart LR
  RULES["Rule list"] --- REASON["Decision and exact reason"] --- PROV["Source refs / owner intent / proposal / tool"]
```

DOT I-3 — EXACT RENDERED GRAPH

```
digraph G {node[shape=record];ui[label="{POLICY / PROVENANCE: decision search | {Rule list: P00 through P12 | DECISION: allow deny approval / exact reason / argument evidence | PROVENANCE: owner intent / source refs / proposal / tool} | Footer: policy version / raw JSON}"];}
```

MERMAID I-4 — EVAL LAB WIREFRAME

```
flowchart LR
  METRICS["Baseline table / n / CI"] --- FAIL["Failed cases / replay diff"] --- MANIFEST["Commit / policy / prompts / model / hardware"]
```

DOT I-4 — EXACT RENDERED GRAPH

```
digraph G {node[shape=record];ui[label="{EVAL LAB: experiment selector / sealed warning | {Baseline comparison: rates / n / CI | Failed cases: filter / diff / replay | MANIFEST: commit / prompts / policy / model / hardware} | Footer: reproducibility status / export}"];}
```

EMPTY CHART J — SECURITY TRADE-OFF SOURCE

Chart.js type scatter. X axis Clean-task success (%), min 0 max 100. Y axis Attack success (%), min 0 max 100. Legend datasets B0 prompt-only, B1 approve-all, B2 policy. ALL dataset.data arrays are EMPTY. Title: TO BE FILLED AFTER MEASUREMENT. B3 is a process comparison, not plotted as a separate runtime defense.

EMPTY CHART K — ROUTING TRADE-OFF SOURCE

Chart.js type bubble. X axis Task quality (%) min 0 max 100. Y axis Cost (USD per successful briefing), min 0. Bubble size represents sensitive cloud bytes. Legend R0 strongest cloud, R1 cheap cloud, R2 local-first, R3 privacy static. ALL dataset.data arrays are EMPTY. Title: TO BE FILLED AFTER MEASUREMENT.

EMPTY CHART L — IDEMPOTENCY SOURCE

Chart.js type bar. Categories Without idempotency, With idempotency. Y axis Duplicate side effects per 1,000 fault-injected attempts. Both dataset values are NULL, not zero. Title: TO BE FILLED AFTER MEASUREMENT.

RENDER INSTRUCTIONS
INFERENCE: To render Mermaid source, paste a diagram block into https://mermaid.live/ and export SVG/PNG. To reproduce exact structural PNGs, send the DOT block to https://quickchart.io/graphviz?format=png&graph=<URL-ENCODED-DOT>. To render Chart.js source, use the documented chart endpoint https://quickchart.io/documentation/usage/parameters/ and the labels/values specified above; keep J–L empty until measured. The PDF already embeds the rendered diagrams, so no external renderer is needed to read it.